

**UNIVERSIDADE FEDERAL DE RORAIMA
CENTRO DE CIÊNCIAS E TECNOLOGIA
CURSO DE CIÊNCIA DA COMPUTAÇÃO**

RAFAEL DA SILVA

JAQUELINE BRAGA MENESES

**RELATÓRIO TÉCNICO: SUBSISTEMA DE MEMÓRIA, UNIDADE DE
CONTROLE E CAMINHO DE DADOS**

Laboratório de Circuitos - Projeto Integrador

BOA VISTA / RR

2026

RAFAEL DA SILVA

JAQUELINE BRAGA MENESES

RELATÓRIO TÉCNICO: SUBSISTEMA DE MEMÓRIA, UNIDADE DE CONTROLE E CAMINHO DE DADOS

Laboratório de Circuitos - Projeto Integrador

Relatório técnico apresentado como requisito de
avaliação na disciplina Arquitetura e Organização de
Computadores (DCC301) do Curso de Ciência da
Computação da Universidade Federal de Roraima.
Professor: Prof. Dr. Herbert Oliveira Rocha

BOA VISTA / RR

2026

SUMÁRIO

1 Introdução	4
2 Ferramentas e organização dos arquivos	5
3 Parte I — Subsistema de memória	7
3.1 Mapa de memória	7
3.2 Decodificador 2 → 4 e tabela verdade dos CS	7
3.3 Componentes base	7
xor2 — XOR com AND, NOT e OR (Componente 03)	7
PARIDADE_IMPARG — gerador de paridade ímpar (Componente 14)	8
MUX4_8 — multiplexador 4 → 1 de 8 bits (Componente 02)	8
MUX4_1 — multiplexador 4 → 1 de 1 bit (Componente 02)	9
FF_D — flip-flop D (Componente 01)	9
FF_JK — flip-flop JK (Componente 01)	10
SOMADOR8 — somador de 8 bits (Componente 08)	11
DEC_7SEG — decodificador de 7 segmentos (Componente 16)	11
Primo4 — detector de número primo de 4 bits (Componente 17)	12
3.4 ROM, RAM e banco de registradores	12
MEMORIA_ROM — ROM 16×8 (Componente 05)	12
MEMORIA_RAM — RAM 16×8 (Componente 06)	13
BANCO_REG — banco de 4 registradores de 8 bits (Componente 07)	13
3.5 Módulo de memória integrado	14
Teste T-01 — seleção de região	16
Teste T-02 — erro de paridade	16
3.6 Memória cache	17
Divisão do endereço	17
Organização do circuito	17
Comparador de rótulos	18
Testes T-03, T-04 e T-05	19
Resultado do traço obrigatório	19
Indicadores	20
Teste de conflito com o detector de primos	20
3.7 Planilha	21
4 Parte II — Unidade de controle e caminho de dados	22
4.1 Estados e diagrama	22
4.2 Tabela de transição	22
4.3 Sinais de controle por estado	22

4.4 Mapas de Karnaugh e equações (Componente 15)	23
4.5 Circuito cabeado	24
4.6 Tabela de tempo e CPI	24
4.7 Microprograma	25
4.8 Formato da instrução e caminho de dados	25
4.9 Testes da Parte II	26
4.10 Comparação entre as duas versões	26
5 Declaração de uso de inteligência artificial	28
6 Divisão do trabalho	29
7 Conclusão	30
Referências	31

1 Introdução

Este relatório documenta o Projeto Integrador do Laboratório de Circuitos da disciplina de Arquitetura e Organização de Computadores. O trabalho foi dividido em duas partes. A Parte I constrói um subsistema de memória de 64 bytes com decodificação de endereços, quatro regiões de 16 bytes e bit de paridade, e uma memória cache de mapeamento direto com quatro linhas, cujo comportamento foi medido sobre um traço obrigatório de leituras. A Parte II projeta a unidade de controle de um processador multiciclo de 8 bits em duas versões, cabeada (com flip-flops D e portas lógicas) e microprogramada (com um microcontador e uma ROM de microcódigo), e liga cada uma a um caminho de dados que executa um pequeno programa de teste.

Os objetivos foram: aplicar a decodificação de endereços e a seleção de dispositivos por multiplexador; implementar e medir uma cache de mapeamento direto, distinguindo faltas compulsórias e faltas por conflito; projetar uma máquina de estados a partir da tabela de transição e de mapas de Karnaugh; e comparar, com números medidos, as abordagens cabeada e microprogramada. Todos os circuitos foram simulados no Logisim-Evolution 5.0.0 e cada um foi verificado com testes manuais, registrados em capturas de tela nas pastas evidencias/ do repositório. Os vetores de teste do próprio simulador foram usados apenas como conferência complementar.

2 Ferramentas e organização dos arquivos

Foi utilizado o Logisim-Evolution versão 5.0.0. Cada arquivo .circ reúne, como subcircuitos, os componentes de que precisa, de modo que abre sozinho, sem bibliotecas externas. Para abrir, basta usar Arquivo → Abrir no Logisim-Evolution; o circuito principal de cada arquivo já está definido. A Tabela 1 resume os arquivos entregues e a Tabela 2 indica onde cada um dos dezessete componentes foi instanciado.

Tabela 1 — Arquivos entregues

Arquivo	Conteúdo
parte1/parte1_memoria.circ	Módulo de memória de 64 bytes (MEMORIA_PARTE1, com túneis, e MEMORIA_PARTE1_FIOS, com fios diretos) e todos os componentes base
parte1/parte1_cache.circ	Cache de mapeamento direto (CACHE_PARTE1) organizada em subcircuitos
parte1/parte1_planilha.xlsx	Traço de 16 acessos, indicadores e teste de conflito
parte2/parte2_cabeada.circ	CPU_CABEADA: unidade de controle cabeada (UC_CABEADA) ligada ao caminho de dados (CAMINHO_DADOS); também contém o DETECTOR_101
parte2/parte2_microprogramada.circ	CPU_MICRO: unidade de controle microprogramada (UC_MICRO, com μ PC CONTADOR_UPC e ROM de microcódigo) ligada ao mesmo caminho de dados
parte2/microcodigo.txt	Conteúdo da ROM de microcódigo: 04 22 90 99 5D
parte2/tabela_tempo.xlsx	Tabela de tempo e cálculo do CPI
componentes/	Arquivos dos componentes individuais
parte1/evidencias/ e parte2/evidencias/	Capturas de tela dos testes T-01 a T-09, uma pasta por teste
parte1/vetores/ e parte2/vetores/	Vetores de teste do Logisim-Evolution (conferência complementar)
relatorio/	Este relatório

Tabela 2 — Onde cada componente foi instanciado

Nº	Componente	Arquivo / subcircuito onde foi usado
01	Flip-flop D e JK	FF_D e FF_JK (componentes); FF_D em VALIDADE, CONTADOR_3, CONTADOR_4 (cache) e nas duas unidades de controle
02	Multiplexador de 4 entradas	MUX4_8 em MEMORIA_PARTE1, BANCO_REG e DADOS_CACHE; MUX4_1 em VALIDADE e ROTULOS; três MUX4_8 no CAMINHO_DADOS (PCSource, ALUSrcA e ALUSrcB)
03	XOR com AND, NOT e OR	xor2 em PARIDADE_IMPAR, MEMORIA_PARTE1 e COMPARADOR_ROTULO
04	Somador com constante 4	SOMA_MAIS_4 no CAMINHO_DADOS ($PC4 = PC + 4$)
05	ROM de 8 bits	MEMORIA_ROM (região 0x00–0x0F), ROM de programa do CAMINHO_DADOS e ROM de microcódigo da UC_MICRO
06	RAM de 8 bits	MEMORIA_RAM (região 0x10–0x1F) e RAM 16×1 de paridade
07	Banco de registradores	BANCO_REG (região 0x20–0x2F) e banco R0–R3 do CAMINHO_DADOS

Nº	Componente	Arquivo / subcircuito onde foi usado
08	Somador de 8 bits	SOMADOR8 em CACHE_PARTE1 (endereço efetivo = ADDR + DESLOC) e dentro da ULA8
09	Detector de "101"	DETECTOR_101 e circuito de teste TESTE_DETECTOR_101 (parte2_cabeada.circ)
10	ULA de 8 bits	ULA8 e CONTROLE_ULA no CAMINHO_DADOS
11	Extensor de sinal 4 → 8	EXT_SINAL_4_8 no CAMINHO_DADOS (imediate do beq)
12	Máquina de estados com portas	UC_CABEADA
13	Contador síncrono	CONTADOR_UPC (µPC da UC_MICRO); CONTADOR_3 e CONTADOR_4 da cache
14	Paridade ímpar	PARIDADE_IMPAR em MEMORIA_PARTE1
15	Mapas de Karnaugh	Seção 4.4 deste relatório (equações da UC_CABEADA)
16	Decodificador de 7 segmentos	DEC_7SEG / DEC_7SEG_S na cache e nas unidades de controle
17	Detector de primo (4 bits)	Primo4 em CACHE_PARTE1 (teste de conflito)

3 Parte I - Subsistema de memória

3.1 Mapa de memória

O barramento de endereços tem 6 bits (A5..A0), o que dá 64 posições, e o barramento de dados tem 8 bits. Os dois bits mais significativos, A5 e A4, escolhem a região; os quatro restantes, A3..A0, escolhem uma das 16 posições dentro dela (Tabela 3).

Tabela 3 — Mapa de memória e equações de seleção

Faixa	Dispositivo	Tamanho	Sinal de seleção
0x00 – 0x0F	ROM de programa	16 bytes	$CS_ROM = A5' \cdot A4'$
0x10 – 0x1F	RAM de dados	16 bytes	$CS_RAM = A5' \cdot A4$
0x20 – 0x2F	Banco de registradores	16 bytes	$CS_REG = A5 \cdot A4'$
0x30 – 0x3F	Região de E/S	16 bytes	$CS_IO = A5 \cdot A4$

3.2 Decodificador 2 → 4 e tabela verdade dos CS

O decodificador foi feito com duas portas NOT (A5' e A4') e quatro portas AND de duas entradas. A Tabela 4 mostra que, em cada combinação de A5 e A4, exatamente um sinal de seleção fica ativo, o que garante que só um dispositivo responde por vez. Para juntar as saídas dos quatro dispositivos foi usado o multiplexador do Componente 02 (MUX4_8), comandado pelos mesmos bits A5 e A4.

Tabela 4 — Tabela verdade do decodificador de região

A5	A4	CS_ROM	CS_RAM	CS_REG	CS_IO	Região
0	0	1	0	0	0	ROM
0	1	0	1	0	0	RAM
1	0	0	0	1	0	Banco
1	1	0	0	0	1	E/S

3.3 Componentes base

Esta seção descreve os componentes construídos pela equipe e usados na Parte I. Para cada um são apresentados os pinos, a imagem do circuito e os testes realizados.

xor2 — XOR com AND, NOT e OR (Componente 03)

Implementa $S = X \cdot Y' + X' \cdot Y$ com 2 portas NOT, 2 portas AND e 1 porta OR, sem usar a porta XOR da biblioteca. É a base do gerador de paridade e do comparador de rótulos da cache.

Tabela 5 — Pinos de xor2

Pino	Direção	Bits	Função
x	entrada	1	operando
Y	entrada	1	operando

Pino	Direção	Bits	Função
S	saída	1	x XOR Y

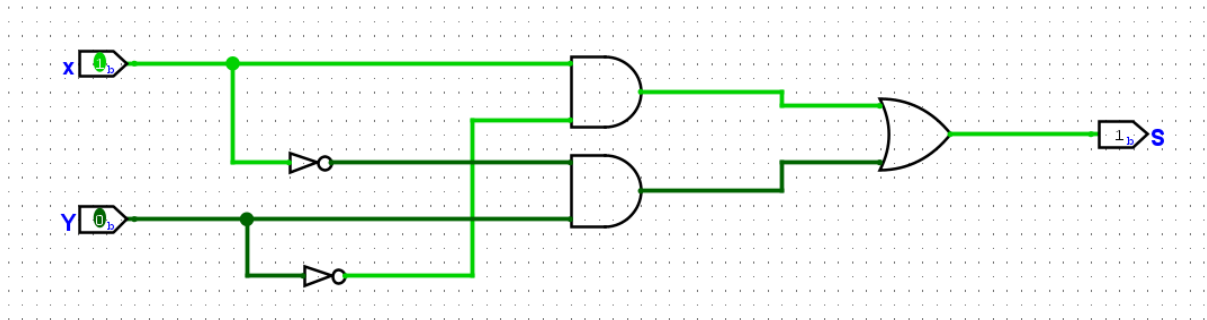


Figura 1 — Circuito xor2 no Logisim-Evolution

Tabela 6 — Testes de xor2

Entradas	Saída esperada	Saída obtida
x=0, Y=0	S=0	S=0
x=0, Y=1	S=1	S=1
x=1, Y=0	S=1	S=1
x=1, Y=1	S=0	S=0

PARIDADE_IMPARGerador de paridade ímpar (Componente 14)

Recebe um byte e produz $P = \text{NOT}(d7 \oplus d6 \oplus \dots \oplus d0)$, usando uma árvore de 7 instâncias do xor2 e uma NOT final. Assim, o total de bits 1 no dado mais o bit P é sempre ímpar.

Tabela 7 — Pinos de PARIDADE_IMPARG

Pino	Direção	Bits	Função
D	entrada	8	dado
P	saída	1	bit de paridade ímpar

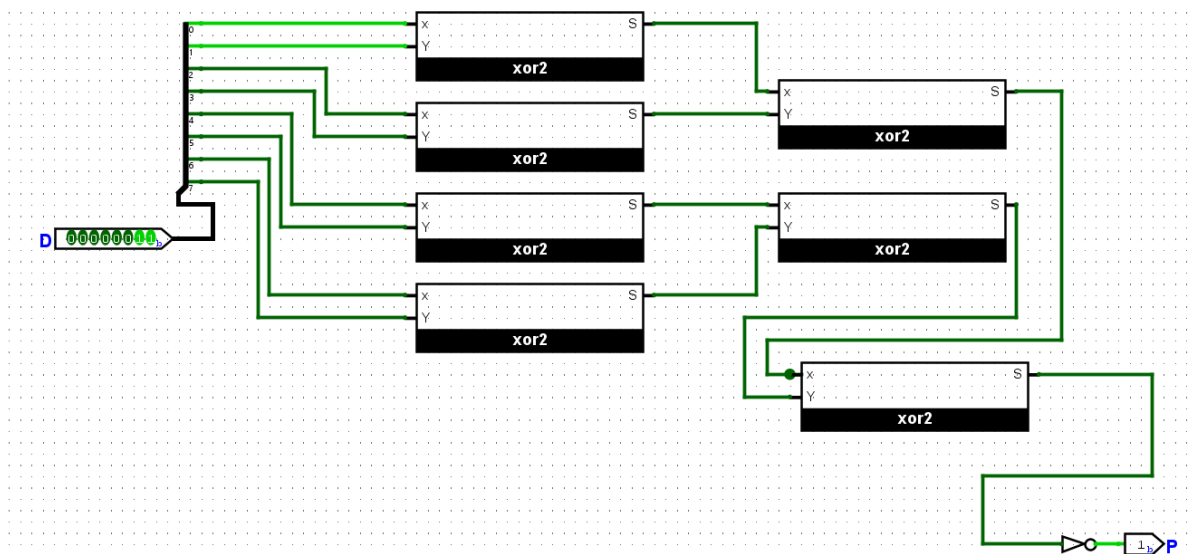


Figura 2 — Circuito PARIDADE_IMPAr no Logisim-Evolution

Tabela 8 — Testes de PARIDADE_IMPAr

Entradas	Saída esperada	Saída obtida
D=0x55 (quatro 1s)	P=1	P=1
D=0x54 (três 1s)	P=0	P=0
D=0x00	P=1	P=1
D=0xFF	P=1	P=1
D=0x01	P=0	P=0

MUX4_8 — multiplexador 4 → 1 de 8 bits (Componente 02)

Decodifica S1S0 em quatro linhas de habilitação (2 NOT e 4 AND), estende cada uma para 8 bits e faz $Y = e0 \cdot I0 + e1 \cdot I1 + e2 \cdot I2 + e3 \cdot I3$ com portas AND de 8 bits e uma OR de 4 entradas.

Tabela 9 — Pinos de MUX4_8

Pino	Direção	Bits	Função
I0..I3	entrada	8	entradas de dados
S1, S0	entrada	1	seleção
Y	saída	8	entrada selecionada

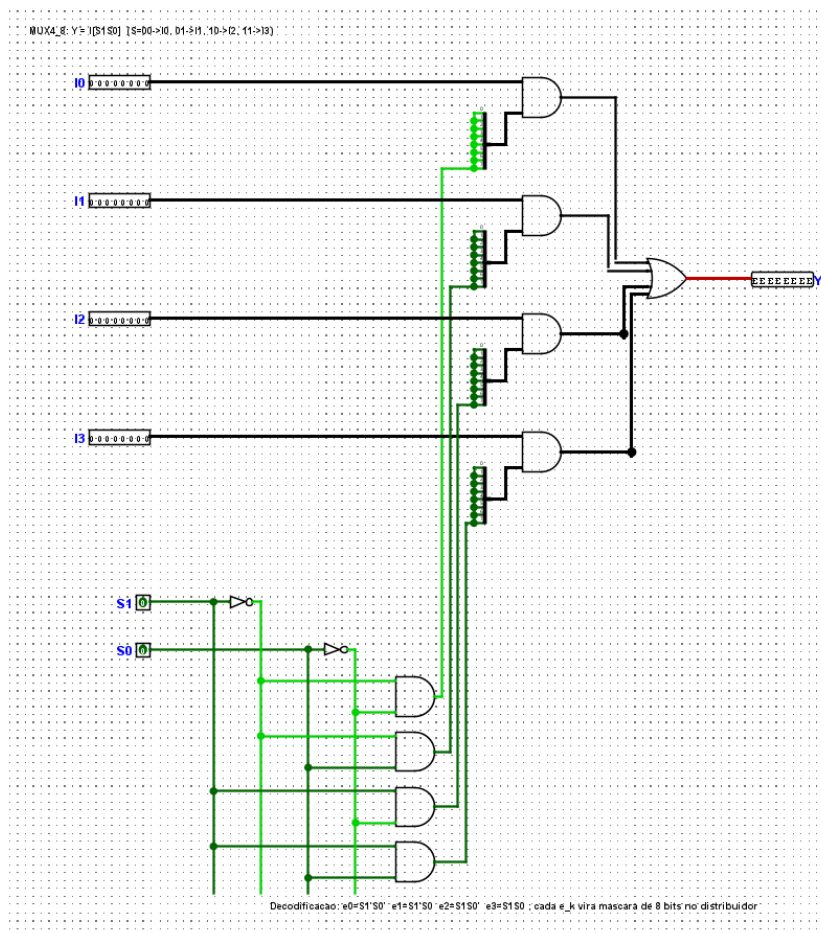


Figura 3 — Circuito MUX4_8 no Logisim-Evolution

Tabela 10 — Testes de MUX4_8

Entradas	Saída esperada	Saída obtida
I0..I3=11,22,33,44; S=00	Y=0x11	Y=0x11
S=01	Y=0x22	Y=0x22
S=10	Y=0x33	Y=0x33
S=11	Y=0x44	Y=0x44

MUX4_1 — multiplexador 4 → 1 de 1 bit (Componente 02)

Versão de 1 bit, feita com 2 NOT, 4 AND de 3 entradas e 1 OR de 4 entradas. É usado na cache para escolher o bit de validade e os bits do rótulo da linha indicada pelo índice.

Tabela 11 — Pinos de MUX4_1

Pino	Direção	Bits	Função
I0..I3	entrada	1	entradas
S1, S0	entrada	1	seleção
Y	saída	1	entrada selecionada

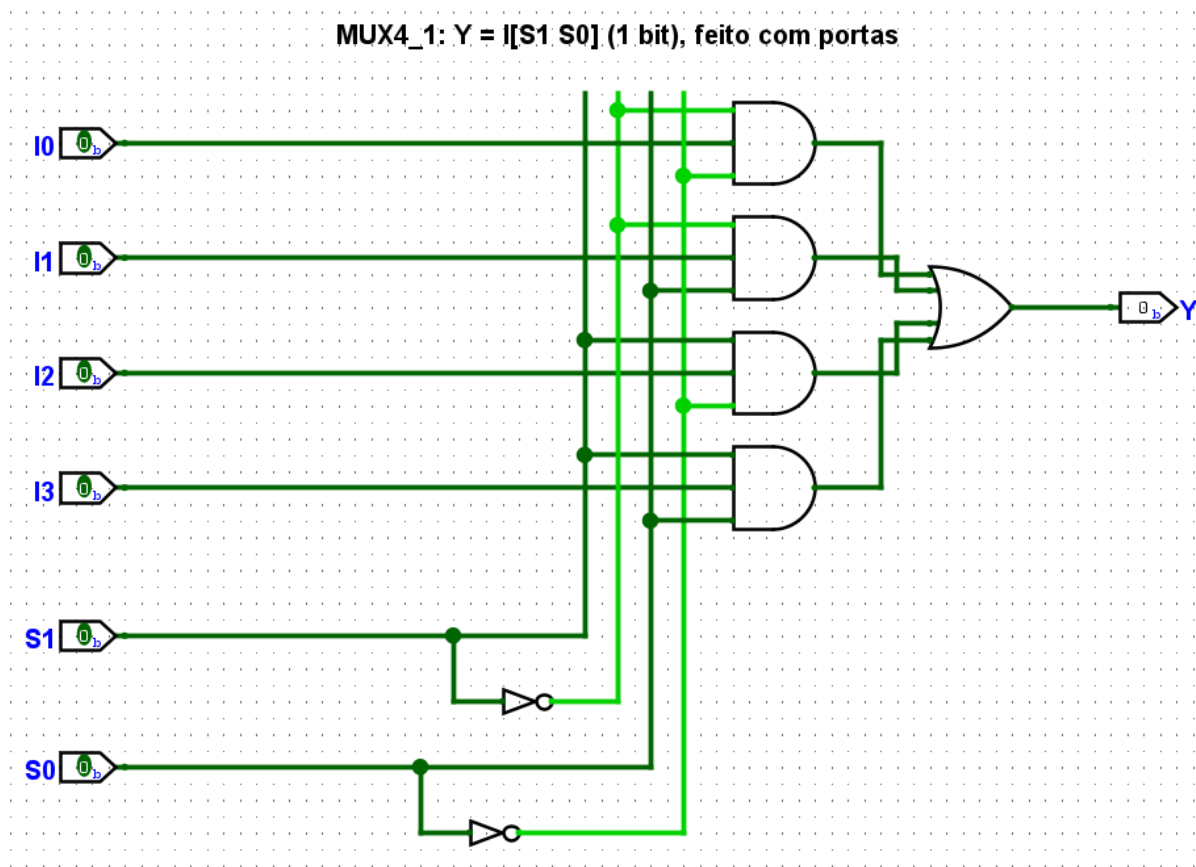


Figura 4 — Circuito MUX4_1 no Logisim-Evolution

Tabela 12 — Testes de MUX4_1

Entradas	Saída esperada	Saída obtida
I0=1, demais 0; S=00	Y=1	Y=1
I1=1; S=01	Y=1	Y=1
I2=1; S=10	Y=1	Y=1
I3=1; S=11	Y=1	Y=1

FF_D — flip-flop D (Componente 01)

Flip-flop D sensível à borda de subida, construído com seis portas NAND, com set e reset assíncronos ativos em nível alto. É usado nos bits de validade da cache, nos contadores e nas duas unidades de controle.

Tabela 13 — Pinos de FF_D

Pino	Direção	Bits	Função
data	entrada	1	dado
clock	entrada	1	relógio (borda de subida)

Pino	Direção	Bits	Função
set	entrada	1	força Q=1
reset	entrada	1	força Q=0
Q, Qn	saída	1	estado e complemento

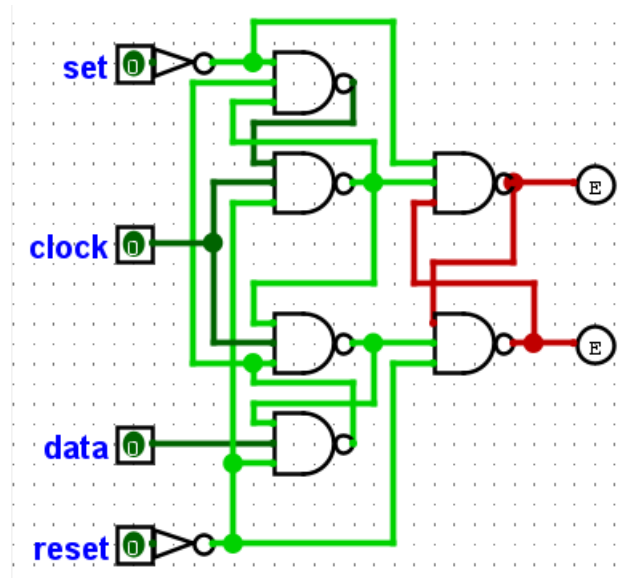


Figura 5 — Circuito FF_D no Logisim-Evolution

Tabela 14 — Testes de FF_D

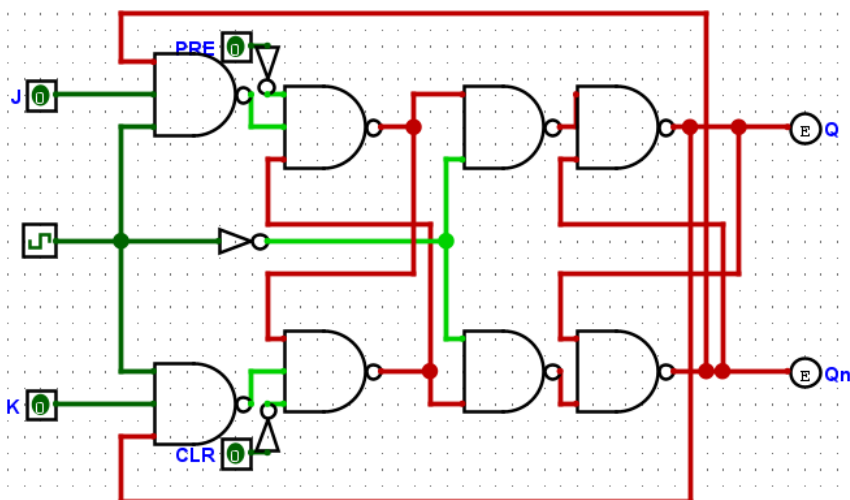
Entradas	Saída esperada	Saída obtida
reset=1 e depois 0	Q=0	Q=0
data=1, clock 0 → 1	Q=1	Q=1
data=0, clock mantido em 1	Q=1 (sensível à borda)	Q=1
data=0, clock 0 → 1	Q=0	Q=0
set=1	Q=1	Q=1

FF_JK — flip-flop JK (Componente 01)

Flip-flop JK mestre-escravo com portas NAND, preset e clear. O mestre copia as entradas com o relógio em 1 e o escravo transfere para Q quando o relógio volta a 0.

Tabela 15 — Pinos de FF_JK

Pino	Direção	Bits	Função
J, K	entrada	1	controle
clock	entrada	1	relógio
PRE, CLR	entrada	1	preset e clear
Q, Qn	saída	1	estado e complemento



Flip-Flop-JK

Este é um flip-flop jk, feito com preset e clear, aonde não podemos ligar ambos ao mesmo tempo.

Figura 6 — Circuito FF_JK no Logisim-Evolution

Tabela 16 — Testes de FF_JK

Entradas	Saída esperada	Saída obtida
J=1, K=0, pulso	Q=1 (set)	Q=1
J=0, K=0, pulso	Q mantém	Q mantém
J=0, K=1, pulso	Q=0 (reset)	Q=0
J=1, K=1, pulso	Q inverte	Q inverte

SOMADOR8 — somador de 8 bits (Componente 08)

Oito somadores completos em cascata, com $S = A \oplus B \oplus Cin$ e $Cout = A \cdot B + Cin \cdot (A \oplus B)$. Na cache calcula o endereço efetivo apresentado ao circuito: $EA = ADDR + DESLOC$.

Tabela 17 — Pinos de SOMADOR8

Pino	Direção	Bits	Função
A, B	entrada	8	parcelas
Cin	entrada	1	vai-um de entrada
S	saída	8	soma
Cout	saída	1	vai-um de saída

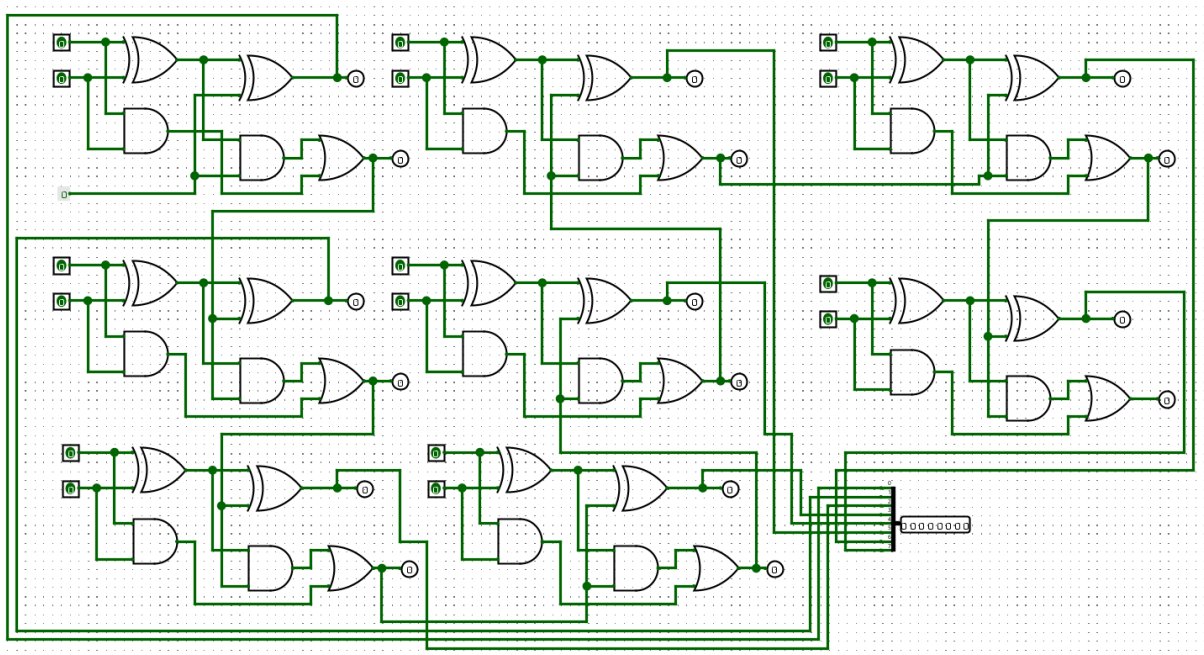


Figura 7 — Circuito SOMADOR8 no Logisim-Evolution

Tabela 18 — Testes de SOMADOR8

Entradas	Saída esperada	Saída obtida
A=5, B=3, Cin=0	S=8, Cout=0	S=8, Cout=0
A=0xFF, B=0x01	S=0x00, Cout=1	S=0x00, Cout=1
A=0x55, B=0xAA	S=0xFF, Cout=0	S=0xFF, Cout=0

DEC_7SEG — decodificador de 7 segmentos (Componente 16)

Converte um valor de 4 bits (A é o mais significativo) nos sete sinais a..g de um display. Os valores de 10 a 15 foram tratados como indiferentes. Mostra o contador de acertos da cache e o estado das unidades de controle.

Tabela 19 — Pinos de DEC_7SEG

Pino	Direção	Bits	Função
A, B, C, D	entrada	1	valor BCD
a..g	saída	1	segmentos

Primo4 — detector de número primo de 4 bits (Componente 17)

$P = A' \cdot B' \cdot C + A' \cdot B \cdot D + B' \cdot C \cdot D + B \cdot C' \cdot D$, obtida por mapa de Karnaugh. Vale 1 para 2, 3, 5, 7, 11 e 13. Na cache gera os endereços do teste de conflito.

Tabela 21 — Pinos de Primo4

Pino	Direção	Bits	Função
A, B, C, D	entrada	1	valor (A é o MSB)
P	saída	1	1 se primo

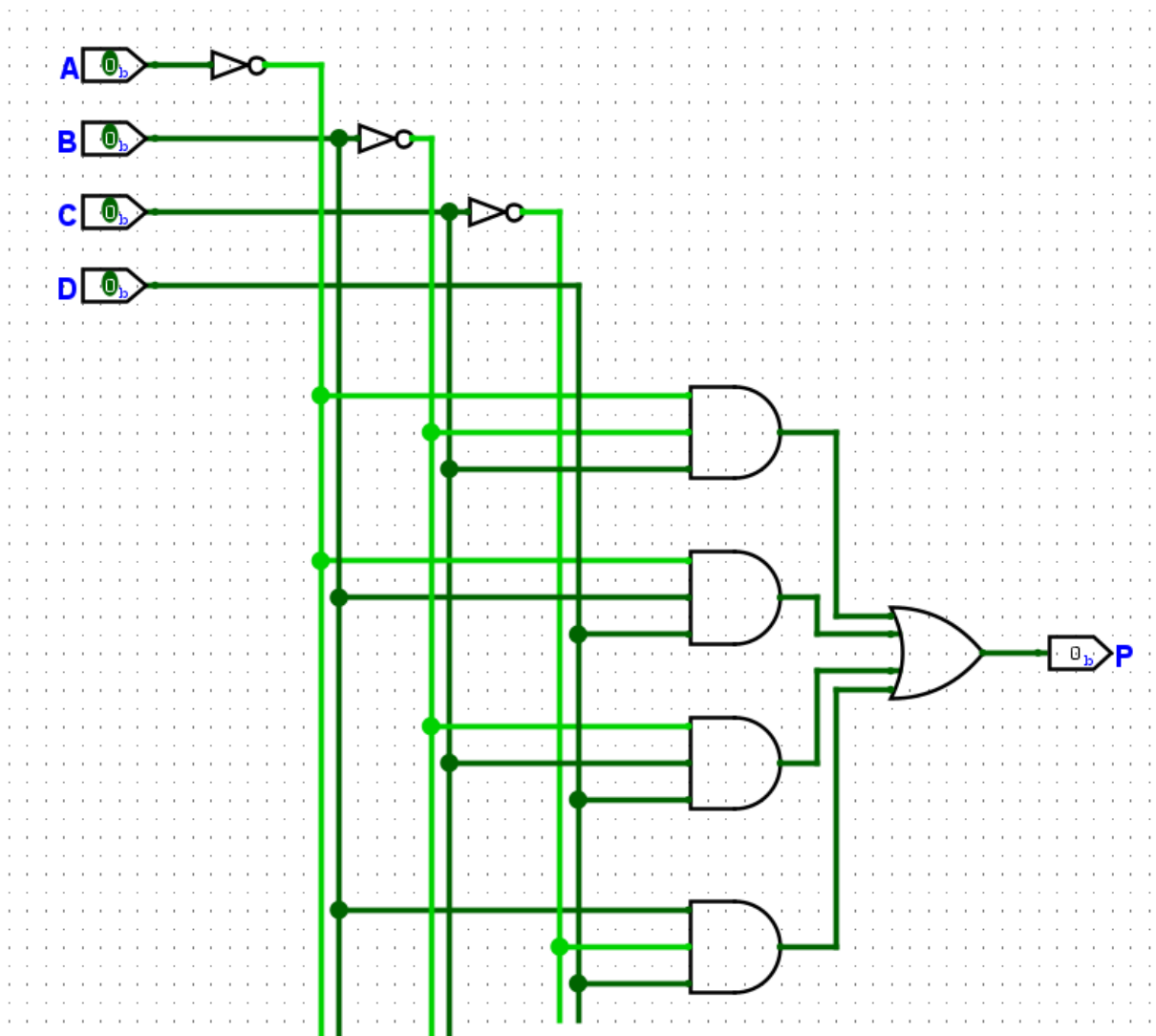


Figura 9 — Circuito Primo4 no Logisim-Evolution

Tabela 22 — Testes de Primo4

Entradas	Saída esperada	Saída obtida
0010 (2)	P=1	P=1
0100 (4)	P=0	P=0
0111 (7)	P=1	P=1
1001 (9)	P=0	P=0
1101 (13)	P=1	P=1

3.4 ROM, RAM e banco de registradores

MEMORIA_ROM — ROM 16×8 (Componente 05)

ROM de 16 posições de 8 bits endereçada por A3..A0. O conteúdo foi preenchido com A0, A1, ..., AF, o que permite reconhecer na saída que o dado veio da ROM.

Tabela 23 — Pinos de MEMORIA_ROM

Pino	Direção	Bits	Função
A	entrada	4	endereço
D	saída	8	dado

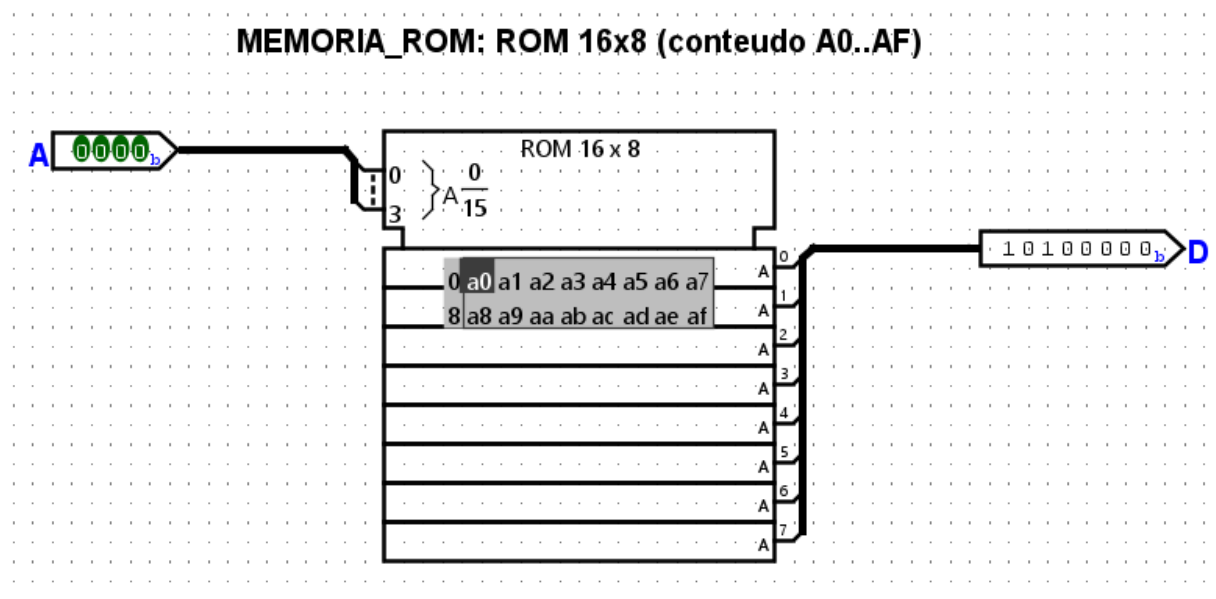


Figura 10 — Circuito MEMORIA_ROM no Logisim-Evolution

Tabela 24 — Testes de MEMORIA_ROM

Entradas	Saída esperada	Saída obtida
A=0000	D=0xA0	D=0xA0

Entradas	Saída esperada	Saída obtida
A=0011	D=0xA3	D=0xA3
A=1111	D=0xAF	D=0xAF

MEMORIA_RAM — RAM 16×8 (Componente 06)

RAM com portas de dados separadas. A escrita ocorre na borda do relógio quando $str = WE \cdot CS$. A leitura também é sincronizada com o relógio, por isso o valor lido aparece após um pulso de CLK.

Tabela 25 — Pinos de MEMORIA_RAM

Pino	Direção	Bits	Função
A	entrada	4	endereço
DIN	entrada	8	dado a gravar
CS	entrada	1	seleção
CLK	entrada	1	relógio
WE	entrada	1	habilita escrita
DOUT	saída	8	dado lido

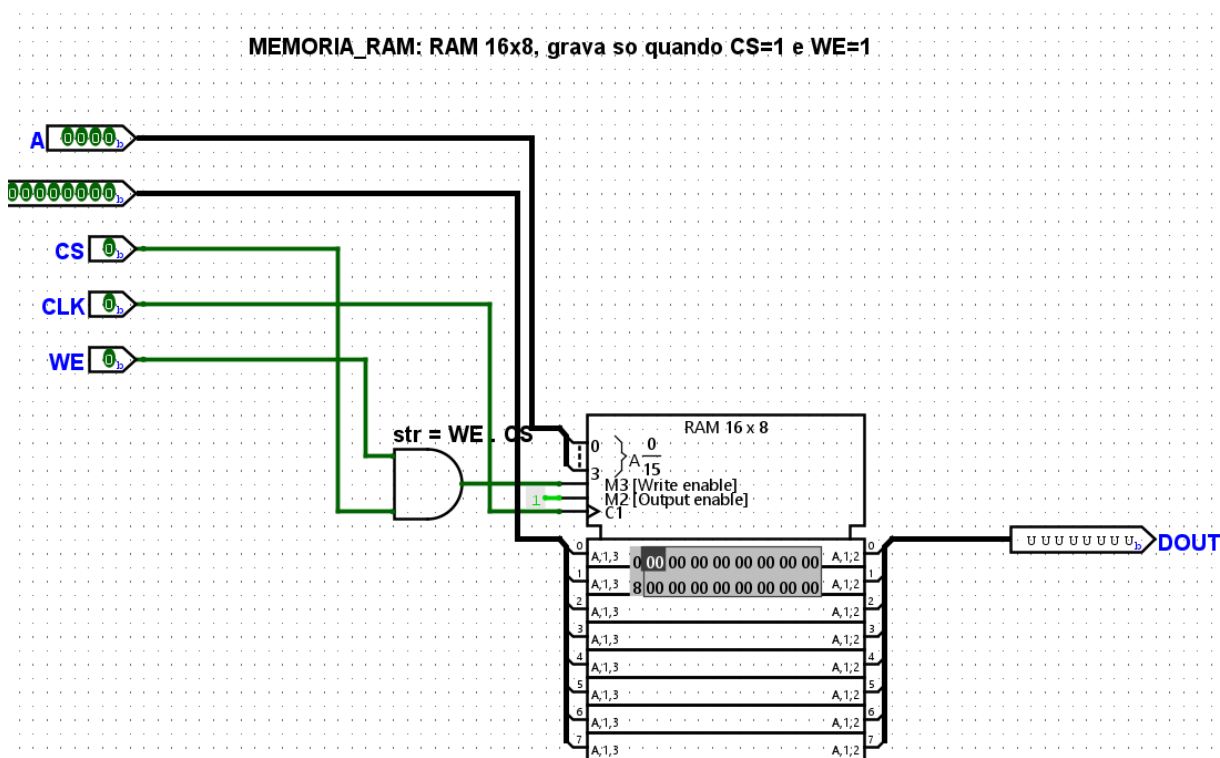


Figura 11 — Circuito MEMORIA_RAM no Logisim-Evolution

Tabela 26 — Testes de MEMORIA_RAM

Entradas	Saída esperada	Saída obtida
CS=1, WE=1, A=2, DIN=55, pulso	DOUT=0x55	DOUT=0x55
WE=0, DIN=FF, pulso	DOUT continua 0x55	DOUT continua 0x55
CS=0, WE=1, A=3, DIN=33, pulso; depois ler A=3	DOUT=0x00 (não gravou)	DOUT=0x00

BANCO_REG — banco de 4 registradores de 8 bits (Componente 07)

Quatro registradores com escrita por decodificador 2→4 (WE·WA) e duas portas de leitura feitas com MUX4_8. Na Parte I a escrita e a leitura usam os bits A1A0 do endereço.

Tabela 27 — Pinos de BANCO_REG

Pino	Direção	Bits	Função
WA1, WA0	entrada	1	registrador de escrita
WE	entrada	1	habilita escrita
WD	entrada	8	dado a gravar
CLK, R	entrada	1	relógio e zera
RA1_x, RA2_x	entrada	1	registradores de leitura
RD1, RD2	saída	8	dados lidos

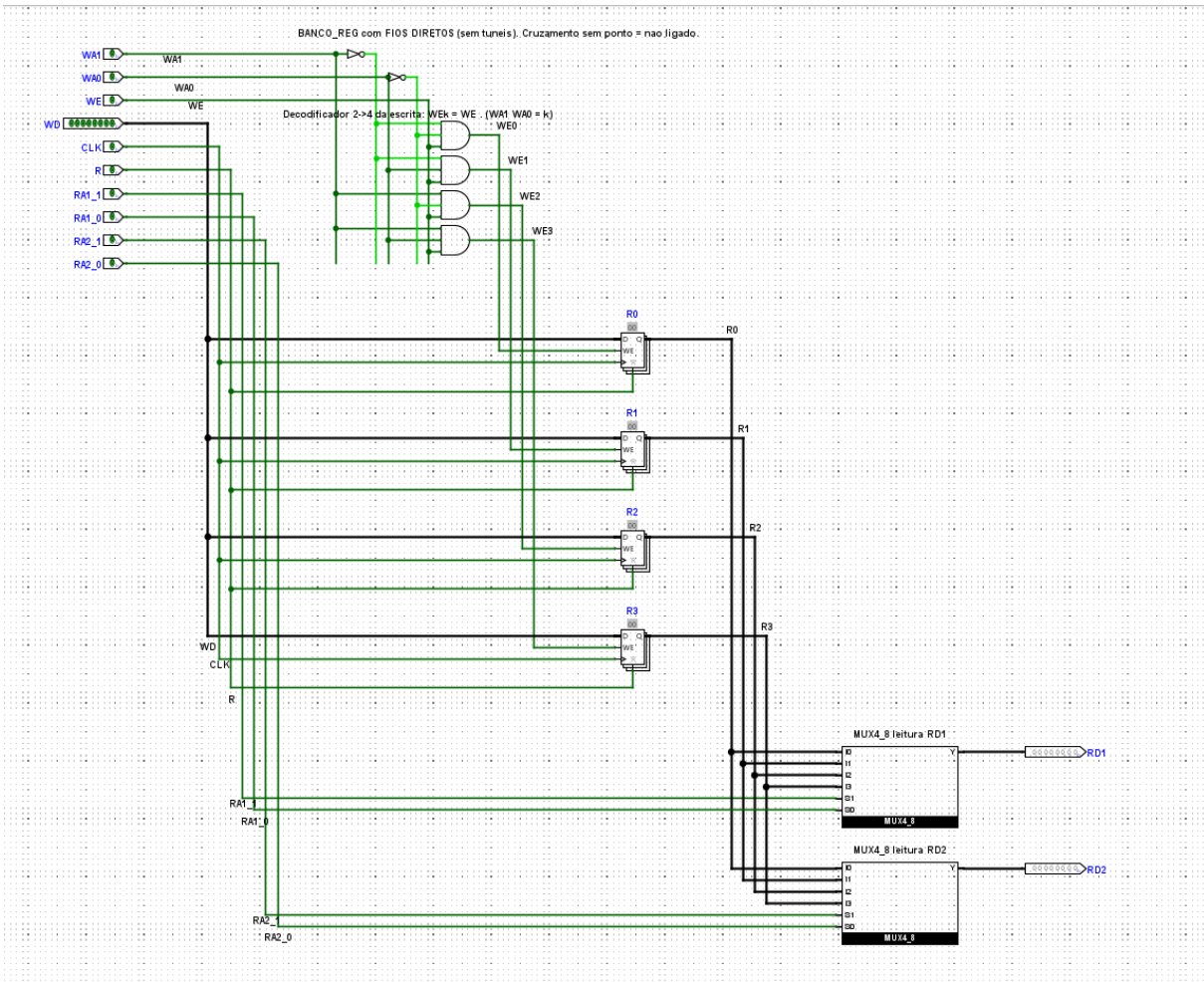


Figura 12 — Circuito BANCO_REG no Logisim-Evolution

Tabela 28 — Testes de BANCO_REG

Entradas	Saída esperada	Saída obtida
R=1 → 0	todos 00	todos 00
grava 11,22,33,44 em R0..R3	R0..R3 = 11,22,33,44	R0..R3 = 11,22,33,44
WE=0, WD=FF, pulso	R0 continua 11	R0 continua 11
RA1=10, RA2=01	RD1=33, RD2=22	RD1=33, RD2=22

3.5 Módulo de memória integrado

O circuito MEMORIA_PARTE1 junta o decodificador de região, a ROM, a RAM, o banco de registradores, a região de E/S (constante 00) e o MUX4_8 de saída comandado por A5 e A4. O bit de paridade é gerado por um PARIDADE_IMPARG no momento da escrita e guardado numa RAM separada de 16×1. Na leitura, a paridade do dado lido é recalculada por um segundo PARIDADE_IMPARG e comparada com a guardada por um xor2. O sinal $ERRO = (P_{calc} \oplus P_{lido}) \cdot CS_{RAM}$ acende o LED de erro.

Foram feitas duas versões equivalentes. A versão MEMORIA_PARTE1 usa túneis para manter cada bloco legível (Figura 13). A versão MEMORIA_PARTE1_FIOS liga tudo com fios diretos, em trilhos verticais, para que se possa seguir cada sinal da entrada à saída (Figuras 14 e 15). As duas passaram nos mesmos testes.

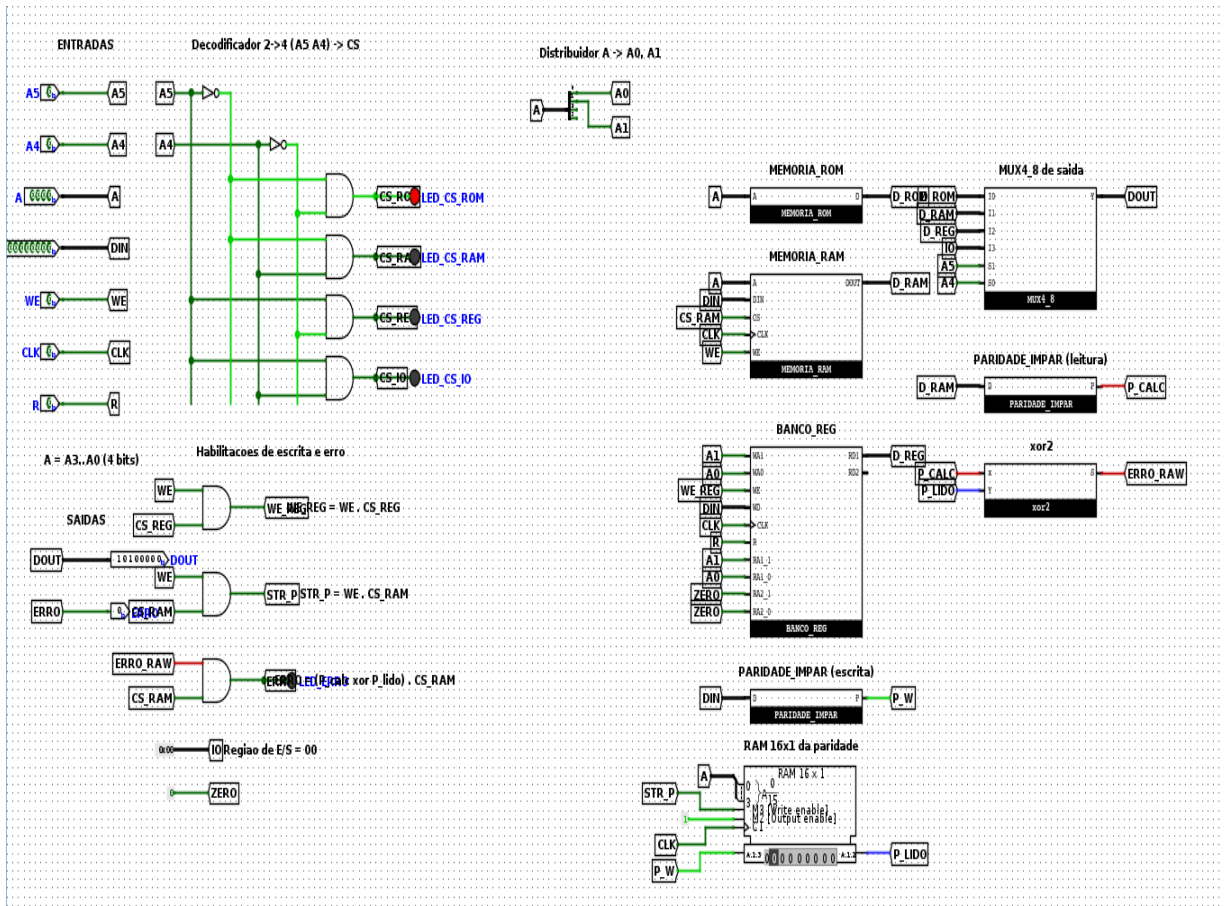


Figura 13 — MEMORIA_PARTE1 (versão com túneis)

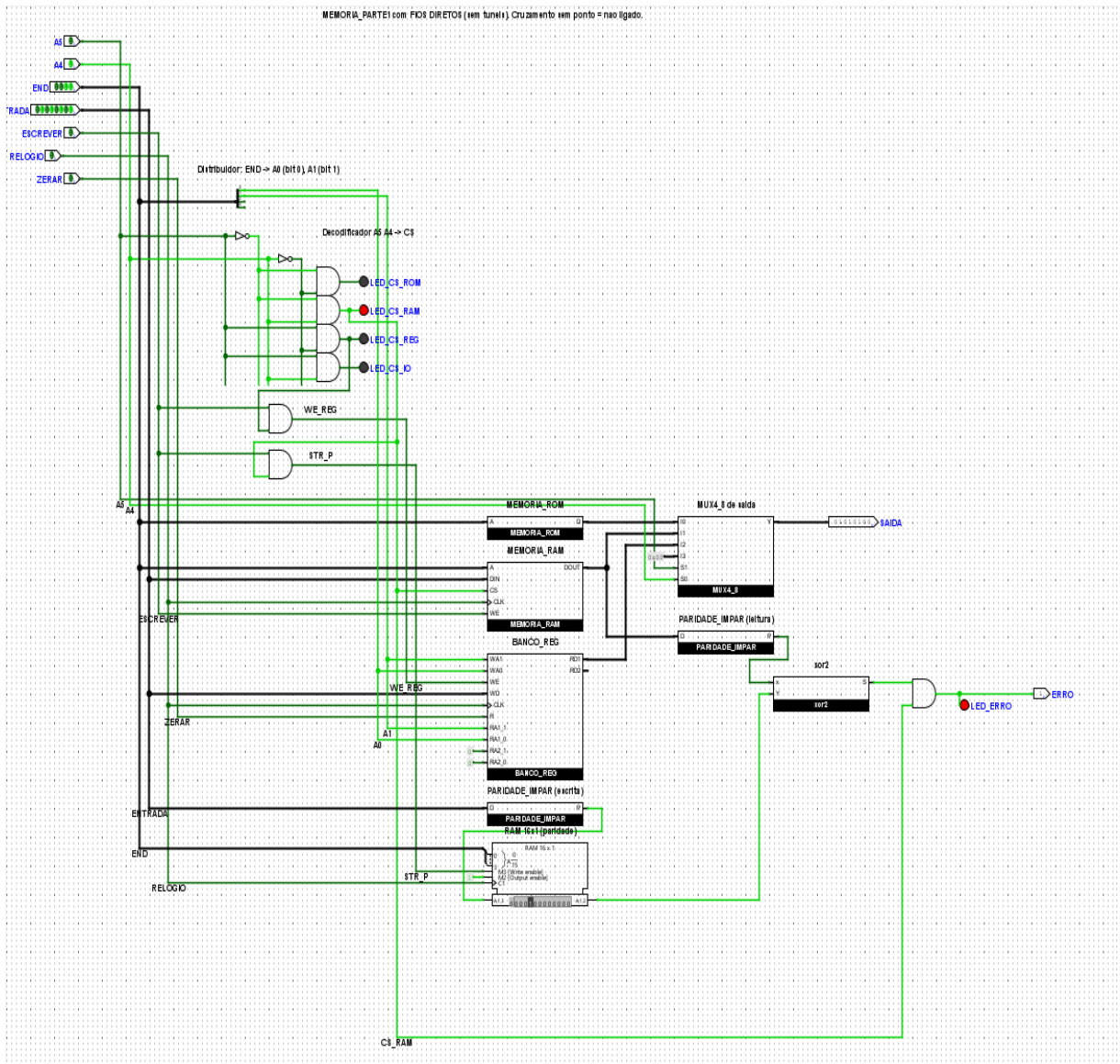


Figura 14 — MEMORIA_PARTE1_FIOS (versão com fios diretos)

parte1_memoria — MEMORIA_PARTE1 desenhada com fios diretos (sem túneis)
 fio grosso preto = barramento de 8/14 bits · fio verde = 1 bit · ponto = ligação · cruzamento sem ponto = não ligado

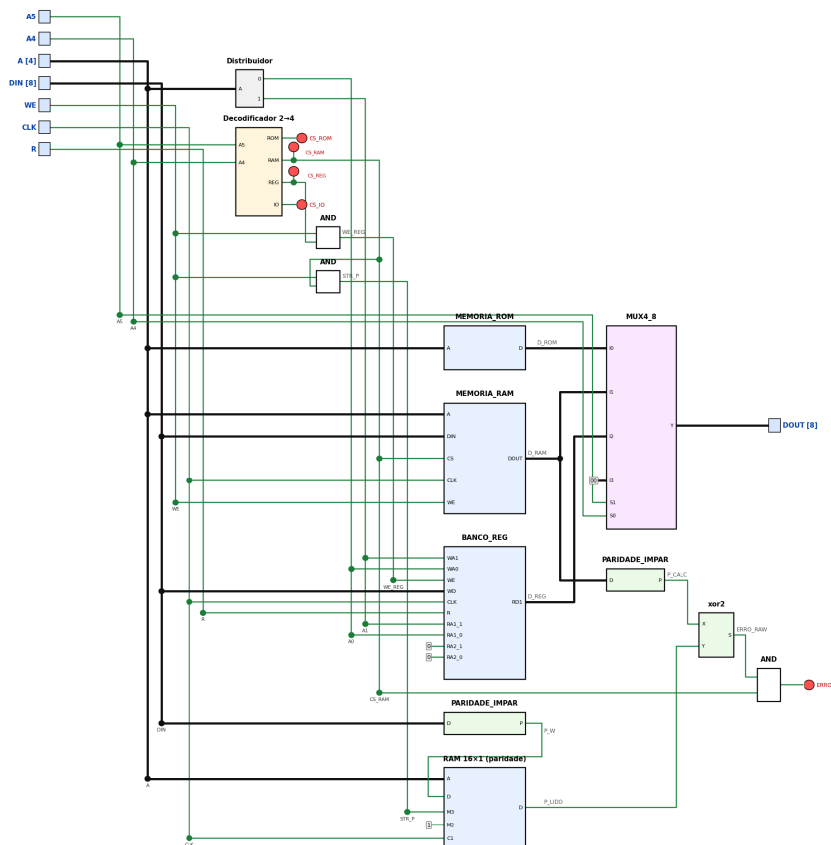


Figura 15 — Diagrama da arquitetura do módulo de memória com fios diretos

Teste T-01 — seleção de região

Preparação: R=1 → 0; gravar 0x44 no banco (A5A4=10, A=0011, WE=1, pulso) e 0x55 na RAM (A5A4=01, A=0011, WE=1, pulso); depois WE=0. Em seguida, com A=0011, variar A5A4 (Tabela 29).

Tabela 29 — Teste T-01

A5 A4	CS ativo	DOUT esperado	DOUT obtido	Figura
00	CS_ROM	0xA3	0xA3	Figura 16
01	CS_RAM	0x55	0x55	Figura 17
10	CS_REG	0x44	0x44	Figura 18
11	CS_IO	0x00	0x00	Figura 19

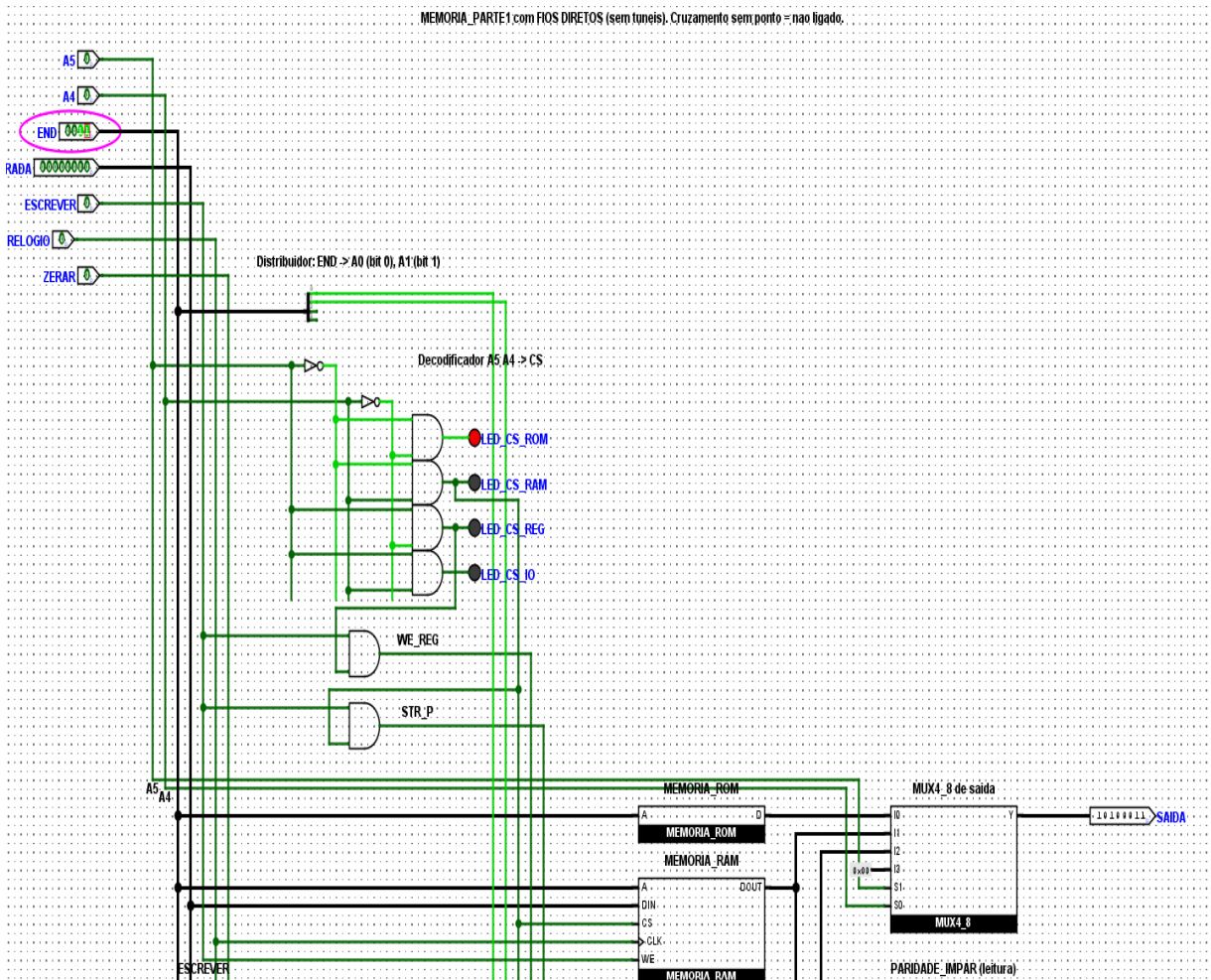


Figura 16 — T-01 com A5A4 = 00 (ROM)

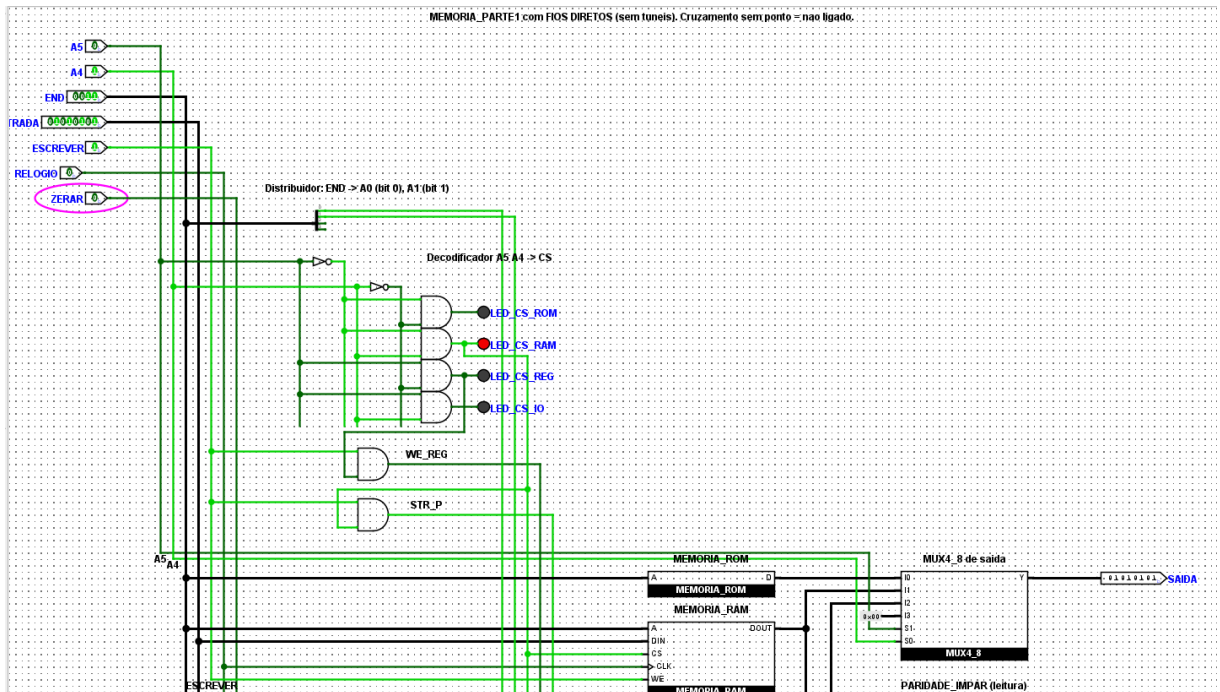


Figura 17 — T-01 com A5A4 = 01 (RAM)

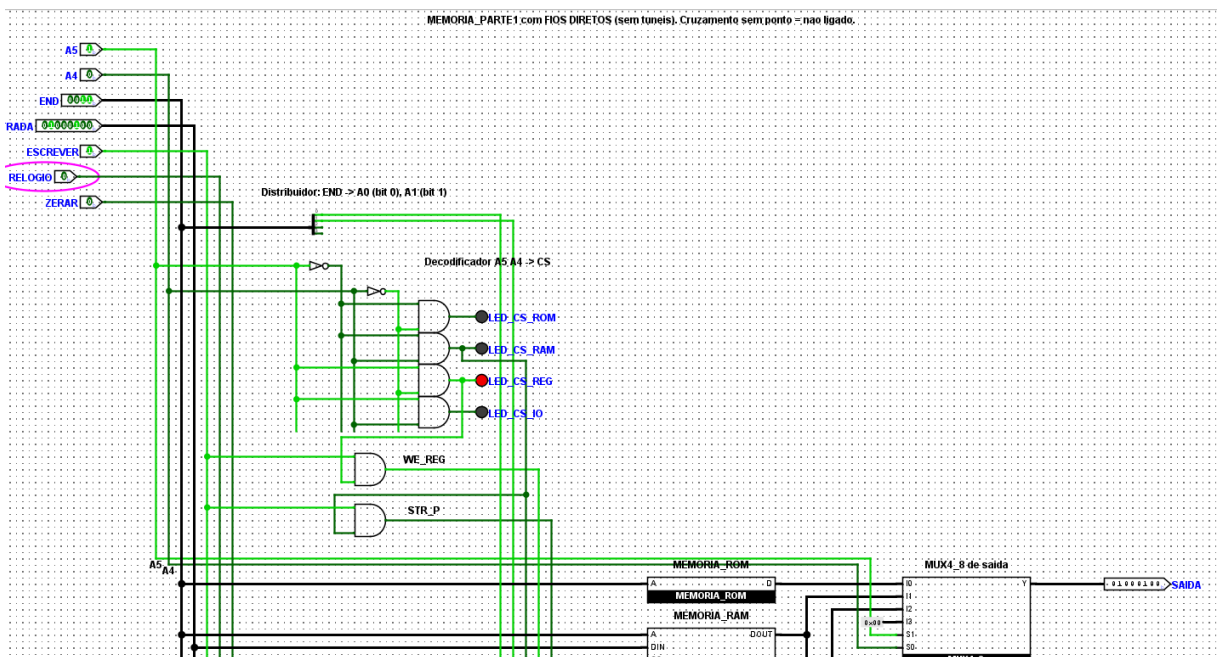


Figura 18 — T-01 com A5A4 = 10 (banco)

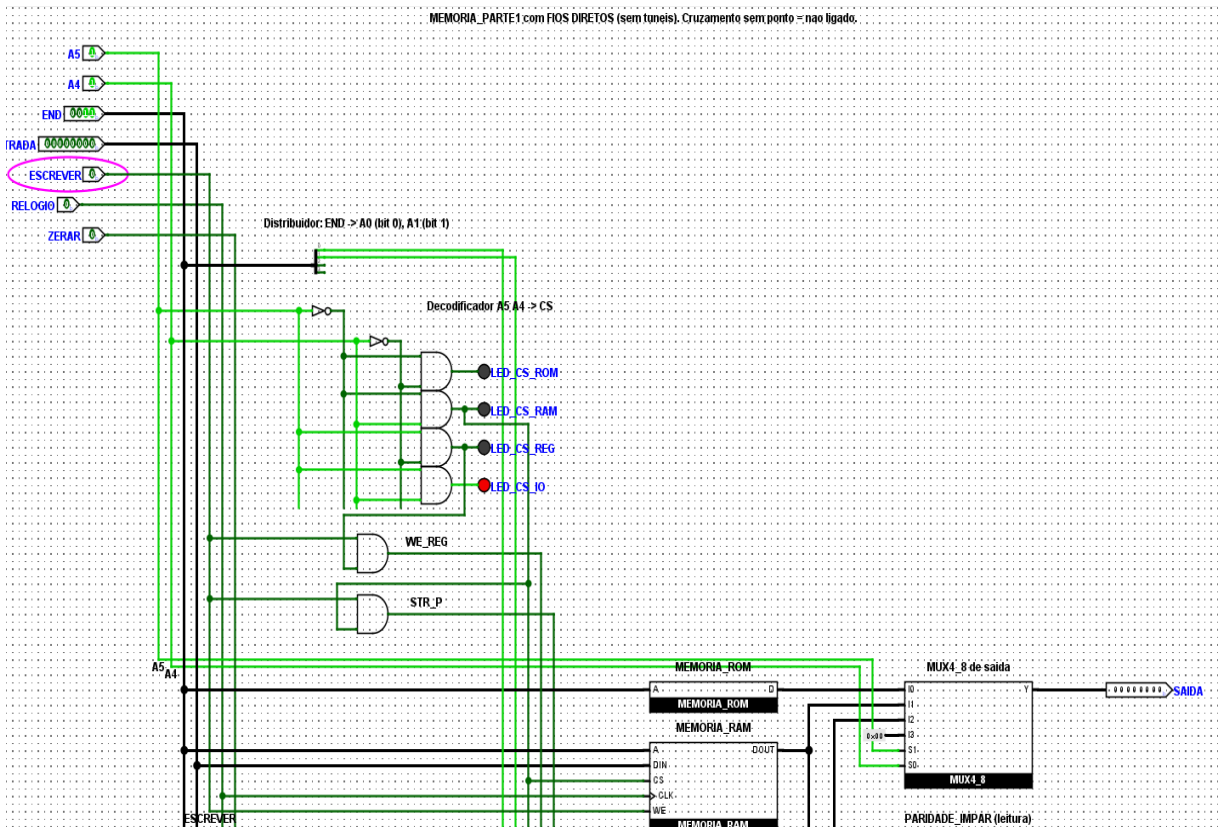


Figura 19 — T-01 com A5A4 = 11 (E/S)

Teste T-02 — erro de paridade

Com A5A4=01 e A=0011, a leitura de 0x55 deixa o LED de erro apagado. Em seguida o conteúdo da posição 3 da RAM foi alterado manualmente para 0x54, simulando a inversão de um bit. Após um pulso de relógio com WE=0, o dado lido passa a ser 0x54 e o LED de erro acende, porque 0x55 tem quatro bits 1 (paridade guardada = 1) e 0x54 tem três bits 1 (paridade recalculada = 0).

Tabela 30 — Teste T-02

Situação	Esperado	Obtido
Leitura de 0x55	DOUT=0x55, ERRO=0	DOUT=0x55, ERRO=0
Após alterar para 0x54	DOUT=0x54, ERRO=1	DOUT=0x54, ERRO=1

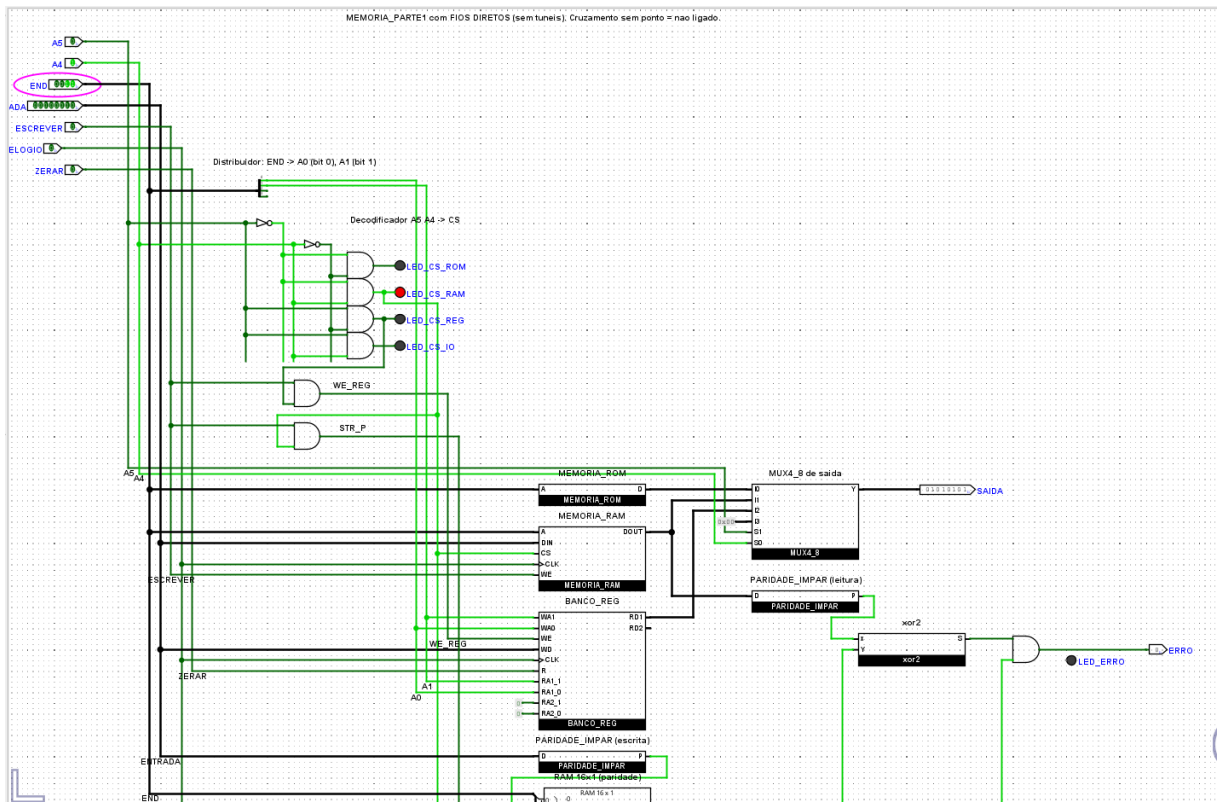


Figura 20 — T-02 antes da alteração (ERRO apagado)

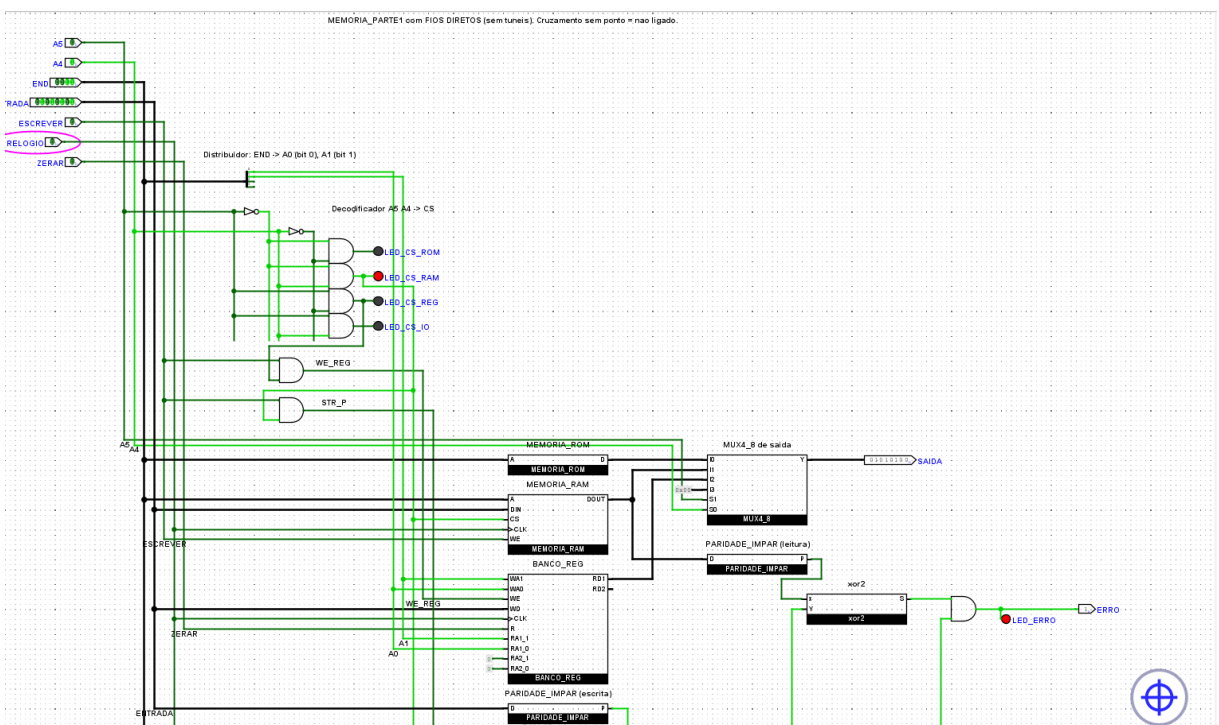


Figura 21 — T-02 depois da alteração (ERRO aceso)

Aprovado: 16 Reprovado 0													
		status	<set>	<seq>	A5	A4	END	ENTRADA	ESCREVER	RELOGIO	ZERAR	SAIDA	ERRO
Show	Set aprovac	1	1	0	0	0	00	0	0	0	1	a0	0
Show	Set aprovac	1	2	0	0	0	00	0	0	0	0	a0	0
Show	Set aprovac	1	3	1	0	3	44	1	0	0	0	<DC>	<DC>
Show	Set aprovac	1	4	1	0	3	44	1	1	0	0	<DC>	<DC>
Show	Set aprovac	1	5	1	0	3	44	1	0	0	0	44	0
Show	Set aprovac	1	6	0	1	3	55	1	0	0	0	<DC>	<DC>
Show	Set aprovac	1	7	0	1	3	55	1	1	0	0	<DC>	<DC>
Show	Set aprovac	1	8	0	1	3	55	1	0	0	0	55	0
Show	Set aprovac	1	9	0	1	3	00	0	0	0	0	<DC>	<DC>
Show	Set aprovac	1	10	0	1	3	00	0	1	0	0	<DC>	<DC>
Show	Set aprovac	1	11	0	1	3	00	0	0	0	0	55	0
Show	Set aprovac	1	12	0	0	3	00	0	0	0	0	a3	0
Show	Set aprovac	1	13	1	0	3	00	0	0	0	0	44	0
Show	Set aprovac	1	14	1	1	3	00	0	0	0	0	00	0
Show	Set aprovac	1	15	0	1	3	00	0	1	0	0	<DC>	<DC>
Show	Set aprovac	1	16	0	1	3	00	0	0	0	0	55	0

Figura 22 — Resultado do vetor de teste teste_MEMORIA_PARTE1.txt (16 de 16)

3.6 Memória cache

A cache atende somente leituras e tem parâmetros fixos (Tabela 31).

Tabela 31 — Parâmetros da cache

Parâmetro	Valor
Organização	Mapeamento direto
Número de linhas	4
Tamanho do bloco	2 bytes
Capacidade de dados	8 bytes
Metadados por linha	1 bit de validade + 3 bits de rótulo
Tempo de acerto	1 ciclo
Penalidade de falta	8 ciclos

Divisão do endereço

O endereço de 6 bits divide-se em três campos. Como o bloco tem 2 bytes, o deslocamento precisa de $\log_2(2) = 1$ bit (A0). Como há 4 linhas, o índice precisa de $\log_2(4) = 2$ bits (A2A1). O restante, $6 - 2 - 1 = 3$ bits, é o rótulo (A5A4A3).

deslocamento = $\log_2(2) = 1$ bit -> A0

índice = $\log_2(4) = 2$ bits -> A2 A1

rotulo = $6 - 2 - 1 = 3$ bits -> A5 A4 A3

Exemplo: 0x22 = 100010. Lendo da direita para a esquerda, o deslocamento é 0, o índice é 01 e o rótulo é 100. Portanto esse endereço só pode ocupar a linha 1, onde será registrado o rótulo 100.

Organização do circuito

O circuito principal CACHE_PARTE1 foi organizado em subcircuitos (Figura 23 e Tabela 32). O endereço efetivo é calculado pelo SOMADOR8 como ADDR +

DESLOC (com DESLOC = 0 nos testes). A cada pulso de CLK o acesso é concluído: em caso de falta, a linha recebe o novo rótulo, o bit de validade vai a 1 e o bloco de 2 bytes é copiado da memória principal; em caso de acerto, o contador de acertos soma 1.

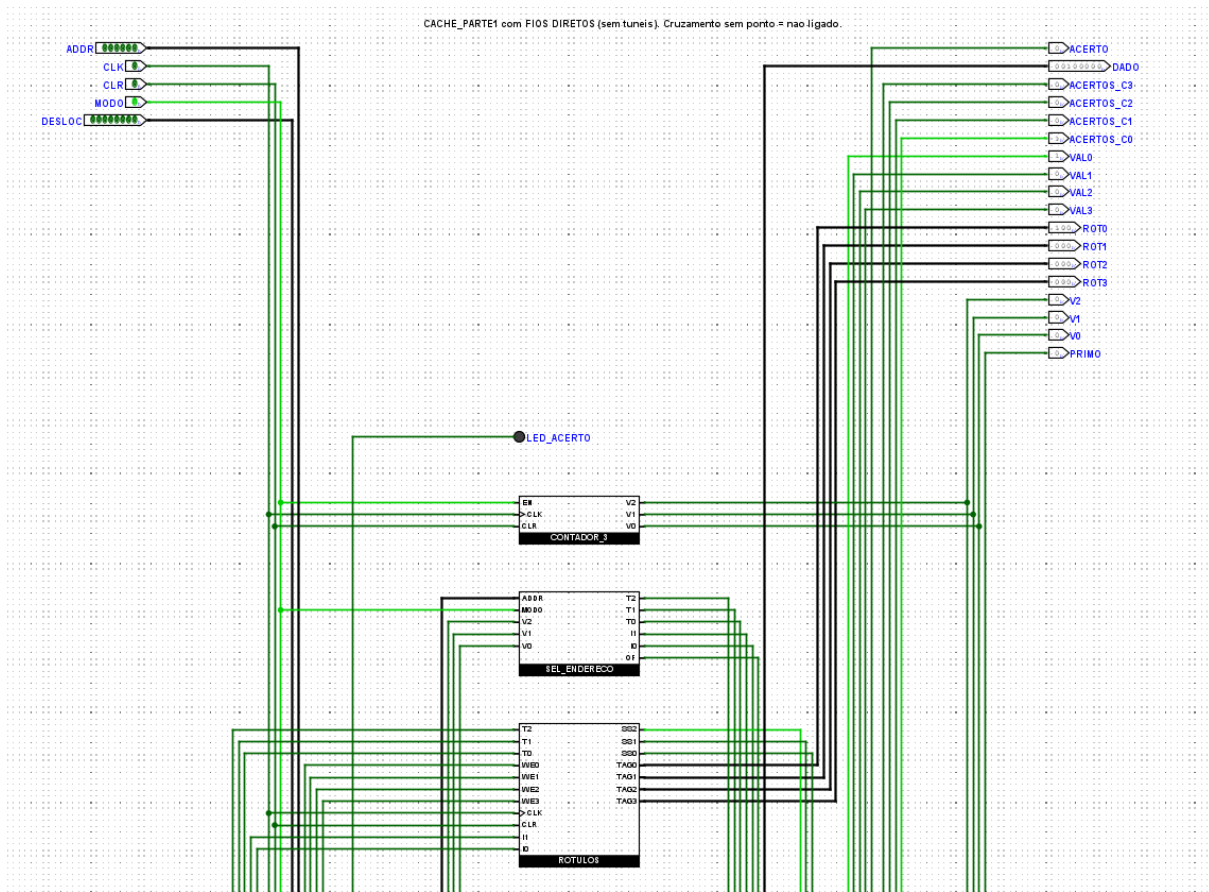


Figura 23 — Circuito principal CACHE_PARTE1 organizado em blocos

Tabela 32 — Subcircuitos da cache

Subcircuito	Função
SEL_ENDERECO	Escolhe o endereço do pino (MODO=0) ou v<<3 (MODO=1) e separa rótulo, índice e deslocamento
ROTULOS	4 registradores de 3 bits e seleção do rótulo da linha com MUX4_1
VALIDADE	4 FF_D zerados por CLR e seleção do bit da linha com MUX4_1
COMPARADOR_ROTULO	3 xor2, OR, NOT e AND: gera ACERTO
CONTROLE_ACESSO	Habilitação, falta e escrita da linha (WE0..WE3)
DADOS_CACHE	Memória principal (ROM de 32 blocos de 2 bytes, conteúdo = endereço), 4 linhas de dados e leitura do byte
CONTADOR_4 / CONTADOR_3	Contador de acertos (4 bits) e contador v (3 bits), feitos com FF_D e portas
DEC_7SEG_S + display	Exibe o número de acertos

Comparador de rótulos

A comparação usa três instâncias do xor2, conforme o enunciado (Figura 24):

$$\text{iguais} = \text{NOT}(\text{(T2 xor S2) OR (T1 xor S1) OR (T0 xor S0)})$$

ACERTO = iguais AND valido[indice]

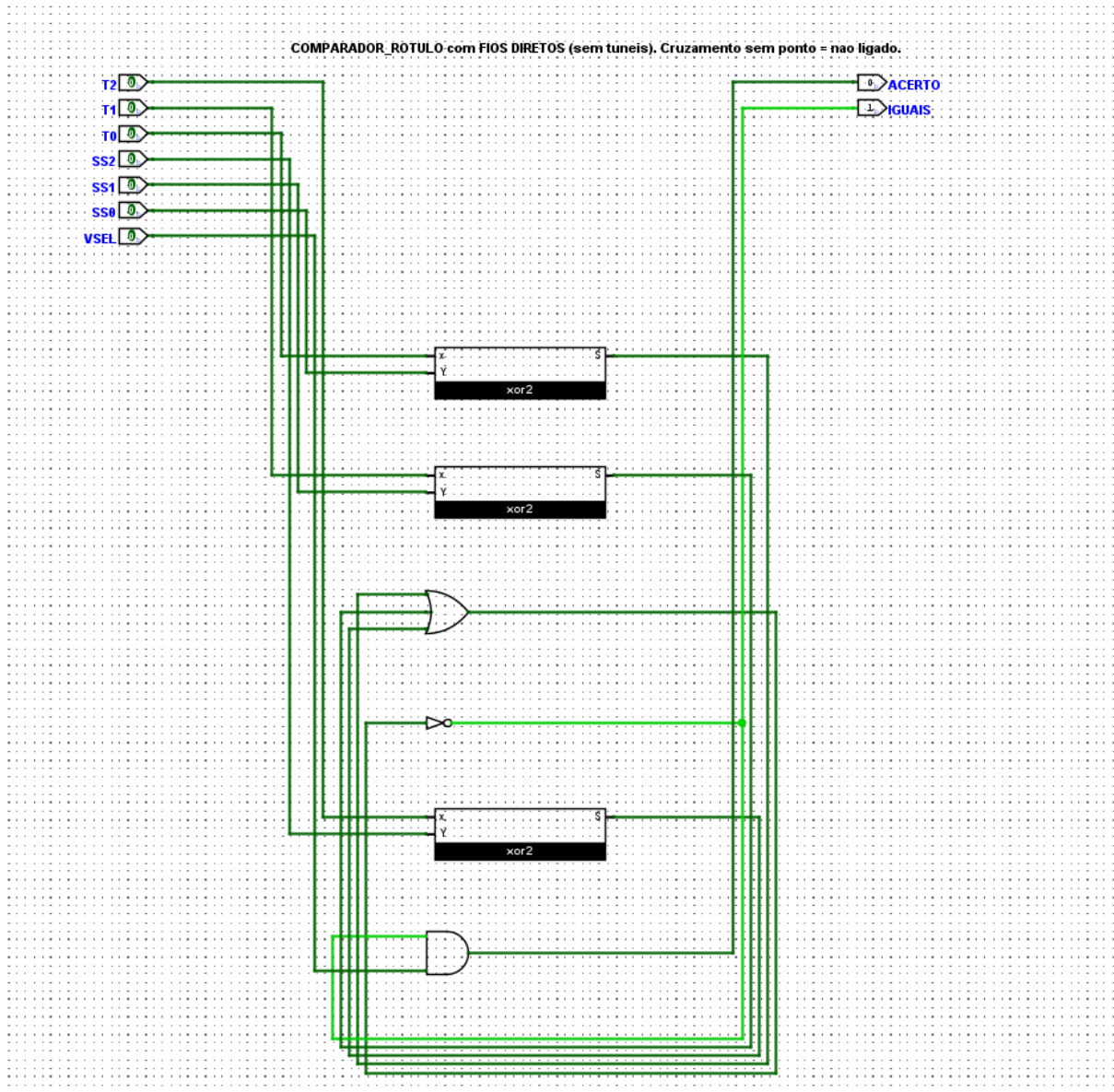


Figura 24 — Subcircuito COMPARADOR_ROTULO

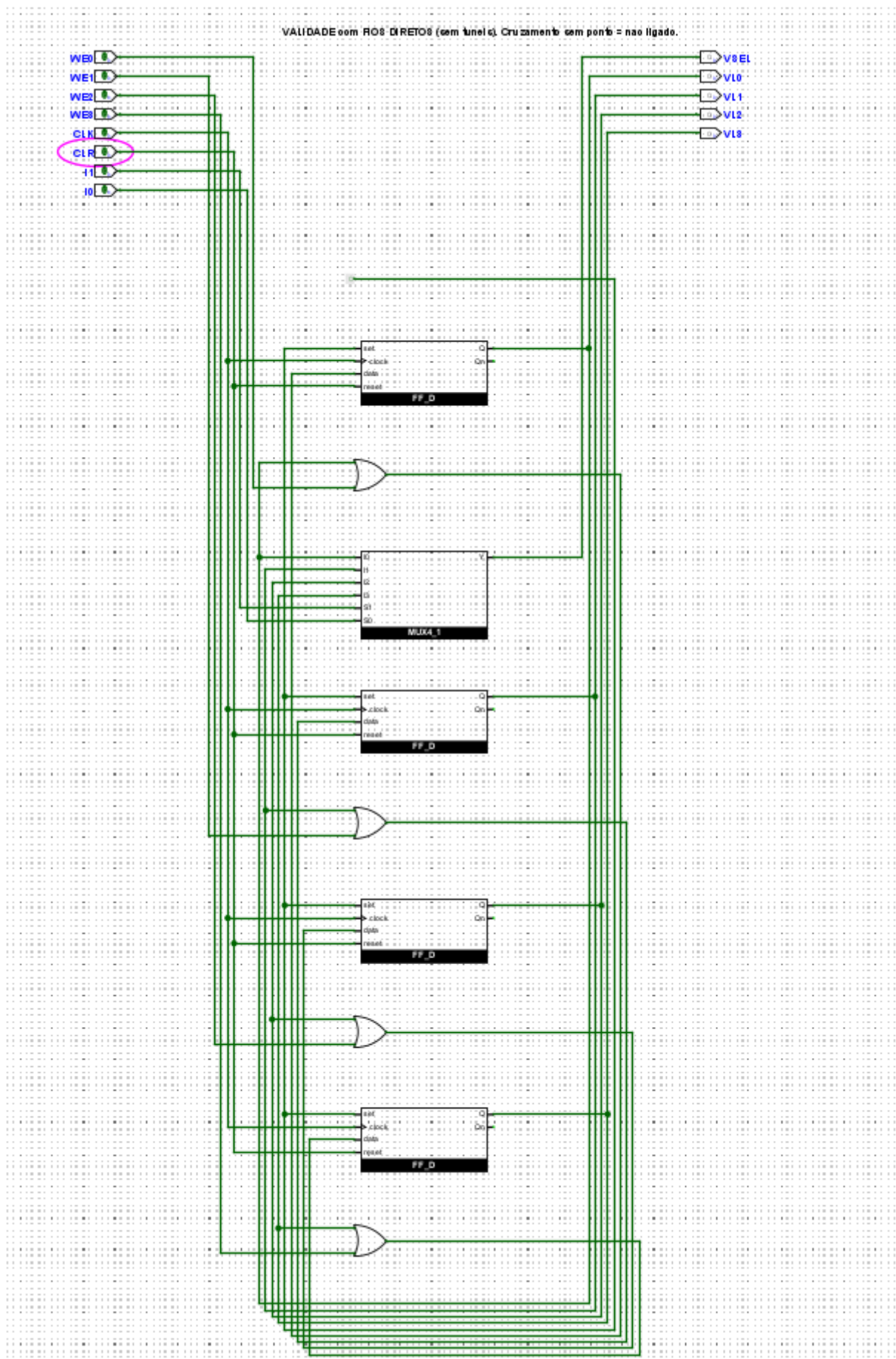


Figura 25 — Subcircuito VALIDADE (bits de validade zeráveis)

Testes T-03, T-04 e T-05

Antes dos testes: Ctrl+R e pulso em CLR, com MODO=0. Em cada acesso, ajusta-se ADDR, lê-se ACERTO e dá-se um pulso em CLK (Tabela 33).

Tabela 33 — Testes T-03 a T-05

Teste	Entrada	Esperado	Obtido	Figura
T-03	Cache zerada, ler 0x00	ACERTO=0 (falta compulsória); após o pulso VAL0=1, ROT0=000	ACERTO=0; após o pulso VAL0=1, ROT0=000	Figura 26
T-04	Ler 0x01	ACERTO=1 (mesmo bloco); display = 1	ACERTO=1; display = 1	Figura 27
T-05	Ler 0x20	ACERTO=0 (conflito); após o pulso ROT0=100	ACERTO=0; após o pulso ROT0=100	Figura 28

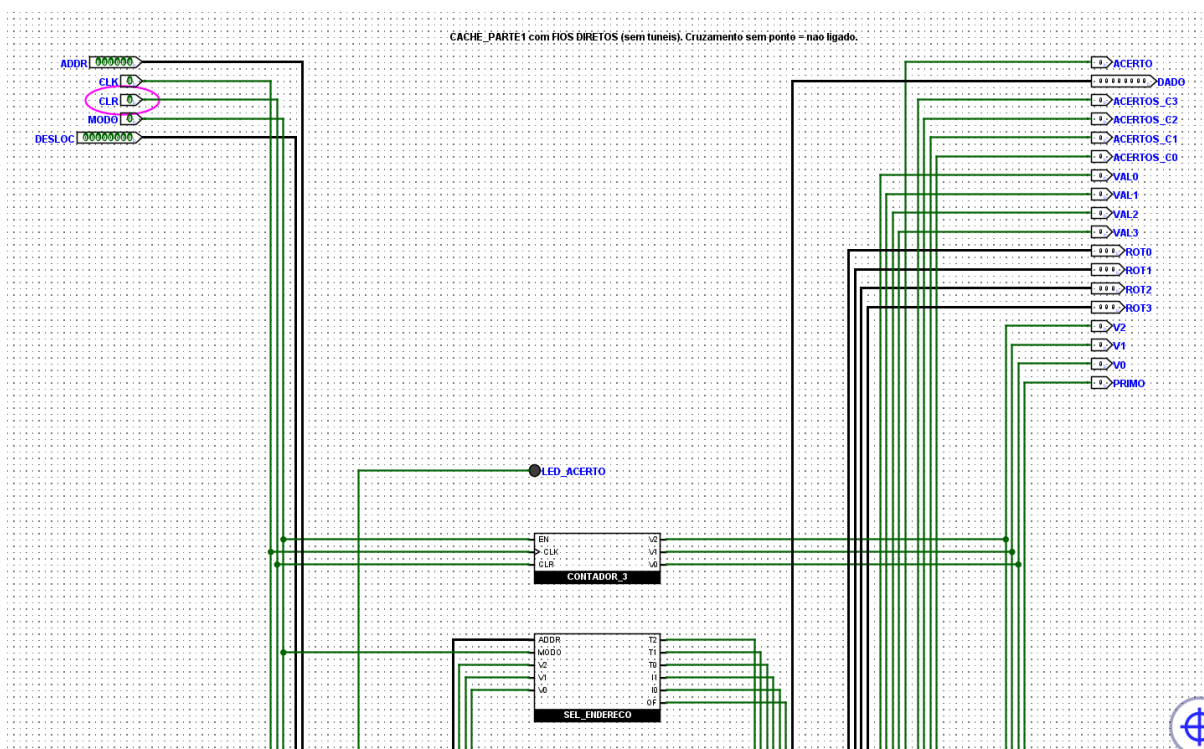


Figura 26 — T-03: falta compulsória ao ler 0x00

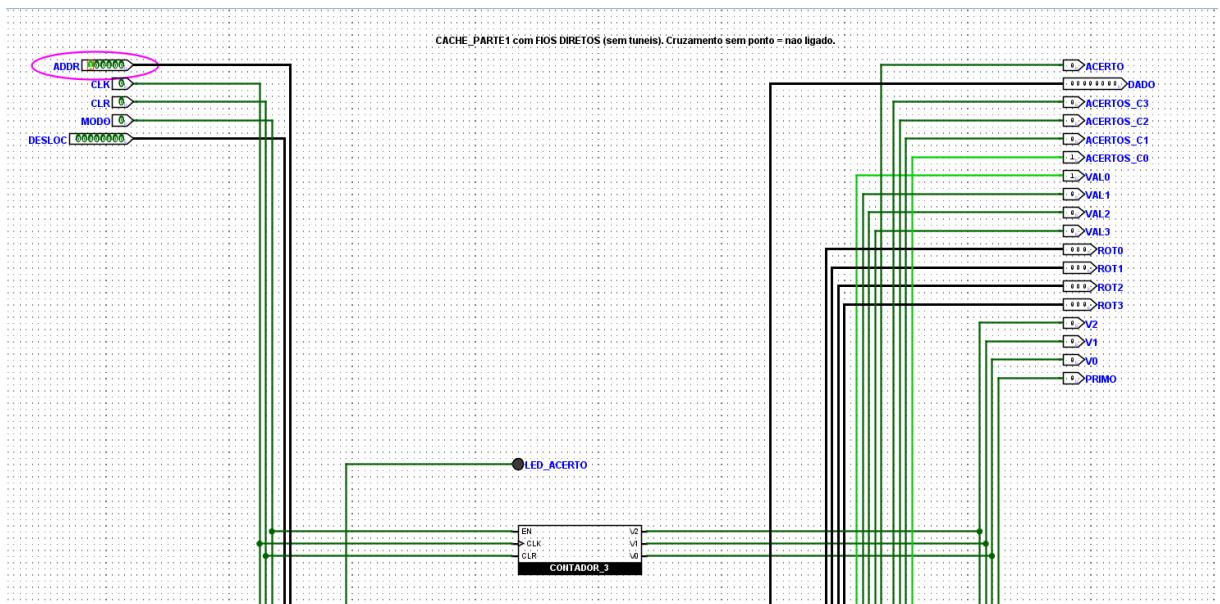
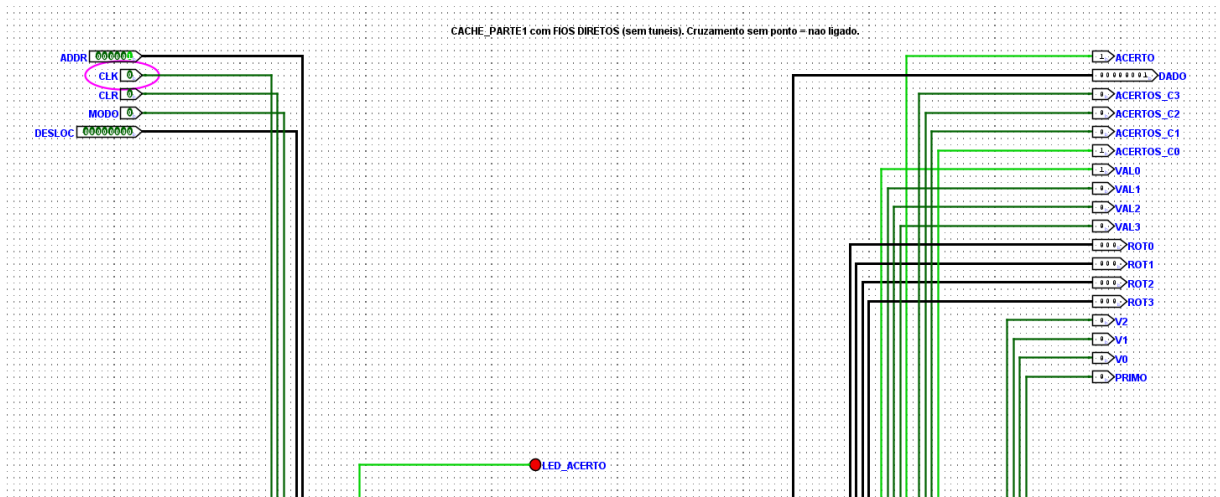


Figura 28 — T-05: falta por conflito ao ler 0x20 (rótulo da linha 0 = 100)

Resultado do traço obrigatório

A Tabela 34 resume o traço de 16 leituras. O mesmo resultado foi obtido na planilha e no circuito: o vetor de teste do traço passou em 50 de 50 verificações e o display terminou mostrando 8 acertos.

Tabela 34 — Resultado do traço de leituras

Nº	Endereço	Rótulo	Índice	Resultado	Tipo de falta
1	0x00	000	00	FALTA	compulsória
2	0x01	000	00	ACERTO	
3	0x02	000	01	FALTA	compulsória
4	0x03	000	01	ACERTO	

Nº	Endereço	Rótulo	Índice	Resultado	Tipo de falta
5	0x00	000	00	ACERTO	
6	0x20	100	00	FALTA	conflito
7	0x21	100	00	ACERTO	
8	0x00	000	00	FALTA	conflito
9	0x04	000	10	FALTA	compulsória
10	0x05	000	10	ACERTO	
11	0x06	000	11	FALTA	compulsória
12	0x07	000	11	ACERTO	
13	0x22	100	01	FALTA	conflito
14	0x23	100	01	ACERTO	
15	0x02	000	01	FALTA	conflito
16	0x03	000	01	ACERTO	

Aprovado: 50 Reprovado 0															
	status	<set>	<seq>	ADDR	CLK	CLR	MODO	ACERTO	DADO	ACERTOS_C3	ACERTOS_C2	ACERTOS_C1	ACERTOS_C0	ROTO	VAL0
Show	Set aprovad	1	4	00	1	0	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>
Show	Set aprovad	1	5	00	0	0	0	1	0000 0000	0	0	0	0	000	<DC>
Show	Set aprovad	1	6	01	0	0	0	1	0000 0001	0	0	0	0	<DC>	<DC>
Show	Set aprovad	1	7	01	1	0	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>
Show	Set aprovad	1	8	01	0	0	0	1	0000 0001	0	0	0	1	000	<DC>
Show	Set aprovad	1	9	02	0	0	0	0	<DC>	0	0	0	1	<DC>	<DC>
Show	Set aprovad	1	10	02	1	0	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>
Show	Set aprovad	1	11	02	0	0	0	1	0000 0010	0	0	0	1	000	<DC>
Show	Set aprovad	1	12	03	0	0	0	1	0000 0011	0	0	0	1	<DC>	<DC>
Show	Set aprovad	1	13	03	1	0	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>
Show	Set aprovad	1	14	03	0	0	0	1	0000 0011	0	0	1	0	000	<DC>
Show	Set aprovad	1	15	00	0	0	0	1	0000 0000	0	0	1	0	<DC>	<DC>
Show	Set aprovad	1	16	00	1	0	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>
Show	Set aprovad	1	17	00	0	0	0	1	0000 0000	0	0	1	1	000	<DC>
Show	Set aprovad	1	18	20	0	0	0	0	<DC>	0	0	1	1	<DC>	<DC>
Show	Set aprovad	1	19	20	1	0	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>
Show	Set aprovad	1	20	20	0	0	0	1	0010 0000	0	0	1	1	100	<DC>
Show	Set aprovad	1	21	21	0	0	0	1	0010 0001	0	0	1	1	<DC>	<DC>
Show	Set aprovad	1	22	21	1	0	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>
Show	Set aprovad	1	23	21	0	0	0	1	0010 0001	0	1	0	0	100	<DC>
Show	Set aprovad	1	24	00	0	0	0	0	<DC>	0	1	0	0	<DC>	<DC>
Show	Set aprovad	1	25	00	1	0	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>
Show	Set aprovad	1	26	00	0	0	0	1	0000 0000	0	1	0	0	000	<DC>
Show	Set aprovad	1	27	04	0	0	0	0	<DC>	0	1	0	0	<DC>	<DC>
Show	Set aprovad	1	28	04	1	0	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>
Show	Set aprovad	1	29	04	0	0	0	1	0000 0100	0	1	0	0	000	<DC>
Show	Set aprovad	1	30	05	0	0	0	1	0000 0101	0	1	0	0	<DC>	<DC>
Show	Set aprovad	1	31	05	1	0	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>
Show	Set aprovad	1	32	05	0	0	0	1	0000 0101	0	1	0	1	000	<DC>
Show	Set aprovad	1	33	06	0	0	0	0	<DC>	0	1	0	1	<DC>	<DC>
Show	Set aprovad	1	34	06	1	0	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>
Show	Set aprovad	1	35	06	0	0	0	1	0000 0110	0	1	0	1	000	<DC>
Show	Set aprovad	1	36	07	0	0	0	1	0000 0111	0	1	0	1	<DC>	<DC>
Show	Set aprovad	1	37	07	1	0	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>
Show	Set aprovad	1	38	07	0	0	0	1	0000 0111	0	1	1	0	000	<DC>

Figura 29 — Conferência complementar do traço: vetor teste_CACHE_traco.txt aprovado em 50 de 50 verificações, terminando com 8 acertos

Indicadores

acertos = 8 faltas = 8 total = 16
taxa de acertos = 8 / 16 = 50% taxa de faltas = 50%
faltas compulsórias = 4 faltas por conflito = 4
AMAT = tempo de acerto + taxa de faltas x penalidade
AMAT = 1 + 0,5 x 8 = 5 ciclos

Teste de conflito com o detector de primos

Com MODO=1, o contador *v* percorre 0 a 7 e, quando o Primo4 indica primo, o endereço *v*<<3 é acessado. Os quatro endereços resultantes têm índice 00 e rótulos diferentes (Tabela 35), por isso disputam a mesma linha e nenhum acerto acontece, mesmo com três linhas ociosas.

Tabela 35 — Teste de conflito

v	Endereço	Binário	Rótulo	Índice	Resultado
2	0x10	010000	010	00	FALTA
3	0x18	011000	011	00	FALTA
5	0x28	101000	101	00	FALTA
7	0x38	111000	111	00	FALTA

A regra de geração explica o resultado: deslocar *v* três posições à esquerda coloca os 3 bits de *v* exatamente em A5A4A3, que é o rótulo, e zera A2A1A0. Como o índice é A2A1, ele é sempre 00. Se a cache tivesse oito linhas, o índice passaria a ter 3 bits (A3A2A1) e o rótulo 2 bits (A5A4). Agora A3 é o bit menos significativo de *v*: 0x10 (*v*=2) iria para a linha 0 e 0x18, 0x28 e 0x38 (*v* ímpar) iriam todos para a linha 4. O conflito diminuiria, mas não desapareceria, pois três endereços ainda disputariam uma única linha.

		status	<set>	<seq>	ADDR	CLK	CLR	MODO	ACERTO	PRIMO	V2	V1	V0	ACERTOS_C0	ACERTOS_C1	ROT0
Show	Set aprovad	1	1	00	0	1	0	0	<DC>	<DC>	0	0	0	0	0	<DC>
Show	Set aprovad	1	2	00	0	0	1	0	<DC>	0	0	0	0	0	0	<DC>
Show	Set aprovad	1	3	00	0	0	1	0	<DC>	0	0	0	0	0	0	<DC>
Show	Set aprovad	1	4	00	1	0	1	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	0	<DC>
Show	Set aprovad	1	5	00	0	0	1	0	<DC>	<DC>	0	0	1	0	0	<DC>
Show	Set aprovad	1	6	00	0	0	1	0	<DC>	0	0	0	1	0	0	<DC>
Show	Set aprovad	1	7	00	1	0	1	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	0	<DC>
Show	Set aprovad	1	8	00	0	0	1	0	<DC>	<DC>	0	1	0	0	0	<DC>
Show	Set aprovad	1	9	00	0	0	1	0	0	1	0	1	0	0	0	<DC>
Show	Set aprovad	1	10	00	1	0	1	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	0	<DC>
Show	Set aprovad	1	11	00	0	0	1	0	<DC>	<DC>	0	1	1	0	0	010
Show	Set aprovad	1	12	00	0	0	1	0	0	1	0	1	1	0	0	<DC>
Show	Set aprovad	1	13	00	1	0	1	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	0	<DC>
Show	Set aprovad	1	14	00	0	0	1	0	<DC>	<DC>	1	0	0	0	0	011
Show	Set aprovad	1	15	00	0	0	1	0	<DC>	0	1	0	0	0	0	<DC>
Show	Set aprovad	1	16	00	1	0	1	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	0	<DC>
Show	Set aprovad	1	17	00	0	0	1	0	<DC>	<DC>	1	0	1	0	0	011
Show	Set aprovad	1	18	00	0	0	1	0	0	1	1	0	1	0	0	<DC>
Show	Set aprovad	1	19	00	1	0	1	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	0	<DC>
Show	Set aprovad	1	20	00	0	0	1	0	<DC>	<DC>	1	1	0	0	0	101
Show	Set aprovad	1	21	00	0	0	1	0	<DC>	0	1	1	0	0	0	<DC>
Show	Set aprovad	1	22	00	1	0	1	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	0	<DC>
Show	Set aprovad	1	23	00	0	0	1	0	<DC>	<DC>	1	1	1	0	0	101
Show	Set aprovad	1	24	00	0	0	1	0	0	1	1	1	1	0	0	<DC>
Show	Set aprovad	1	25	00	1	0	1	0	<DC>	<DC>	<DC>	<DC>	<DC>	<DC>	0	<DC>
Show	Set aprovad	1	26	00	0	0	1	0	<DC>	<DC>	0	0	0	0	0	111

Figura 30 — Teste dos primos: contador *v*, PRIMO e ACERTO = 0 nos quatro acessos

3.7 Planilha

A planilha parte1_planilha.xlsx tem uma linha por acesso e as onze colunas obrigatórias, na ordem pedida (acesso, endereco_hex, endereco_bin, rotulo, indice, deslocamento, valido_antes, rotulo_armazenado_antes, resultado, tipo_falta, rotulo_apos). As colunas C, D, E, F, I e J são calculadas por fórmula a partir da

coluna B, por exemplo =DEC2BIN(HEX2DEC(MID(B2;3;2));6) para o binário e =LEFT(C2;3) para o rótulo. Abaixo do traço estão os indicadores, também por fórmula, e uma segunda aba traz o teste de conflito da Seção 3.6.

A	B	C	D	E	F	G	H	I	J	K
acesso	endereço_hex	endereço_bin	rotulo	indice	deslocamento	valido_antes	rotulo_armazenado_antes	resultado	tipo_falta	rotulo_apos
1	0x00	000000	000	00	0	0 -		FALTA	compulsoria	000
2	0x01	000001	000	00	1	1 000		ACERTO		000
3	0x02	000010	000	01	0	0 -		FALTA	compulsoria	000
4	0x03	000011	000	01	1	1 000		ACERTO		000
5	0x00	000000	000	00	0	1 000		ACERTO		000
6	0x20	100000	100	00	0	1 000		FALTA	conflito	100
7	0x21	100001	100	00	1	1 100		ACERTO		100
8	0x00	000000	000	00	0	1 100		FALTA	conflito	000
9	0x04	000100	000	10	0	0 -		FALTA	compulsoria	000
10	0x05	000101	000	10	1	1 000		ACERTO		000
11	0x06	000110	000	11	0	0 -		FALTA	compulsoria	000
12	0x07	000111	000	11	1	1 000		ACERTO		000
13	0x22	100010	100	01	0	1 000		FALTA	conflito	100
14	0x23	100011	100	01	1	1 100		ACERTO		100
15	0x02	000010	000	01	0	1 100		FALTA	conflito	000
16	0x03	000011	000	01	1	1 000		ACERTO		000
Indicadores (Secao 5.5.2)										
Acertos		8								
Faltas		8								
Total de acessos		16								
Taxa de acertos		50%								
Taxa de faltas		50%								
Faltas compulsorias		4								
Faltas por conflito		4								
Tempo de acerto (ciclos)		1								
Penalidade de falta (ciclos)		8								
AMAT = acerto + taxa_faltas x penalidade (ciclos)		5								

Figura 31 — Print da planilha com fórmulas e indicadores

4 Parte II — Unidade de controle e caminho de dados

4.1 Estados e diagrama

A unidade de controle tem cinco estados, codificados em três flip-flops D (Q2Q1Q0). A entrada B indica o tipo da instrução, obtido do opcode: B=0 para tipo-R (add, sub, and, or) e B=1 para beq. A Figura 32 mostra o diagrama de estados.

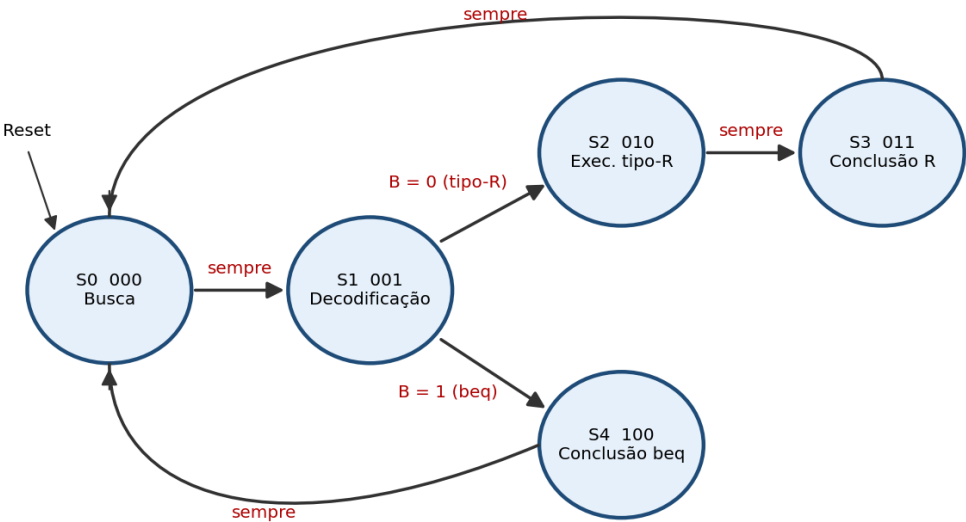


Figura 32 — Diagrama de estados da unidade de controle

4.2 Tabela de transição

Tabela 36 — Tabela de transição

Estado	Q2Q1Q0	B	Próximo N2N1N0
S0 Busca	000	X	001
S1 Decodificação	001	0	010
S1 Decodificação	001	1	100
S2 Exec. tipo-R	010	X	011
S3 Conclusão R	011	X	000
S4 Conclusão beq	100	X	000
não usados	101, 110, 111	X	X (indiferente)

Os estados não usados foram tratados como condições indiferentes. Com as equações obtidas, 101 vai para 100 ou 010, 110 vai para 010 e 111 vai para 000; ou seja, todos voltam a um estado válido em no máximo um ciclo e a máquina não trava. Forçar o retorno a S0 exigiria portas a mais.

4.3 Sinais de controle por estado

Tabela 37 — Sinais de controle gerados em cada estado (valores do circuito)

Estado	PCWr	PCWrC	PCSrc	MemRd	IRWr	RegWr	SrcA	SrcB	ALUOp
S0	1	0	0	1	1	0	0	01	00
S1	0	0	0	0	0	0	0	10	00
S2	0	0	0	0	0	0	1	00	10
S3	0	0	0	0	0	1	1	00	10
S4	0	1	1	0	0	0	1	00	01

Os valores X da tabela de tempo (Seção 4.6) foram fixados de forma que as duas versões gerem exatamente os mesmos sinais ciclo a ciclo, o que permitiu o teste T-09.

4.4 Mapas de Karnaugh e equações (Componente 15)

As Figuras 33 e 34 mostram os mapas das três funções de próximo estado e de quatro sinais de controle, com os agrupamentos. Nos mapas de próximo estado, as linhas são Q2Q1 e as colunas Q0B; os X são os estados não usados. Os contornos tracejados indicam agrupamentos que se completam pela borda do mapa (as linhas 00 e 10 são adjacentes).

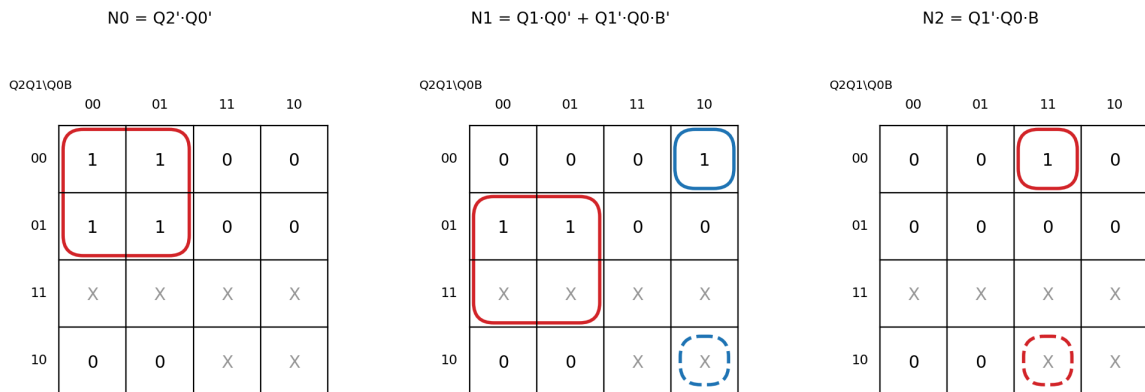


Figura 33 — Mapas de Karnaugh das funções de próximo estado

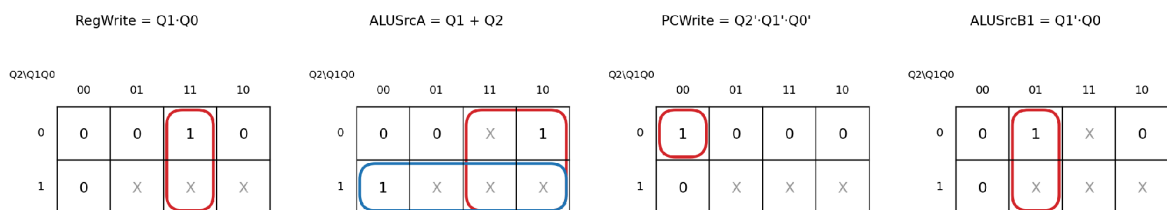


Figura 34 — Mapas de Karnaugh de RegWrite, ALUSrcA, PCWrite e ALUSrcB1

Equações finais em soma de produtos:

$$N0 = Q2' \cdot Q0'$$

$$N1 = Q1 \cdot Q0' + Q1' \cdot Q0 \cdot B'$$

$$N2 = Q1' \cdot Q0 \cdot B$$

$$S0 = Q2' \cdot Q1' \cdot Q0' \rightarrow PCWrite = MemRead = IRWrite = ALUSrcB0$$

$$PCWriteCond = PCSource = ALUOp0 = Q2$$

$$RegWrite = Q1 \cdot Q0 \quad ALUSrcA = Q1 + Q2$$

$$ALUSrcB1 = Q1' \cdot Q0 \quad ALUOp1 = Q1$$

Para ALUSrcB1 a equação mínima seria apenas Q0. Foi escolhida $Q1' \cdot Q0$ para que, no estado S3 (em que o sinal é indiferente), o valor coincida com o da versão microprogramada. O termo $Q1' \cdot Q0$ já é usado em N1 e N2, então não houve custo adicional de portas.

4.5 Circuito cabeadado

O circuito UC_CABEADA usa três FF_D e apenas portas AND, OR e NOT, sem ROM nem tabela verdade da biblioteca. O estado atual aparece num display de sete segmentos através do DEC_7SEG (Figura 35). Contagem de portas: 4 NOT ($Q2'$, $Q1'$, $Q0'$ e B'), 7 AND e 2 OR, totalizando 13 portas.

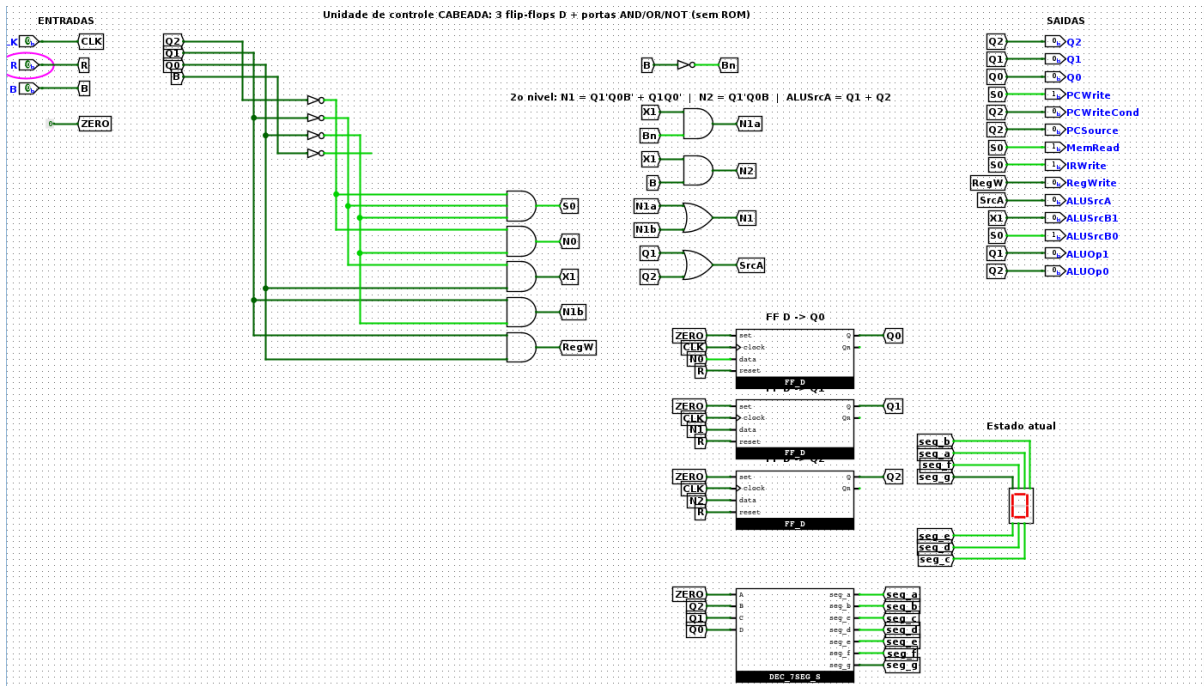


Figura 35 — UC_CABEADA após o reset (display em 0)

4.6 Tabela de tempo e CPI

Tabela 38 — Tabela de tempo

Estado	PCWr	PCWrC	PCSrc	MemRd	IRWr	RegWr	SrcA	SrcB	ALUOp
S0 Busca	1	0	0	1	1	0	0	01	00
S1 Decod.	0	0	X	0	0	0	0	10	00
S2 Exec. R	0	0	X	0	0	0	1	00	10
S3 Concl. R	0	0	X	0	0	1	X	XX	XX
S4 Concl. beq	0	1	1	0	0	0	1	00	01

Justificativa dos X: PCSource não importa quando PCWrite e PCWriteCond valem 0, pois o PC não é escrito. Em S3 o resultado já está em ULAOut e a ULA não é usada, por isso ALUSrcA, ALUSrcB e ALUOp são indiferentes.

Tipo-R: S0, S1, S2, S3 = 4 ciclos

beq: S0, S1, S4 = 3 ciclos

$$CPI = 0,7 \times 4 + 0,3 \times 3 = 2,8 + 0,9 = 3,7$$

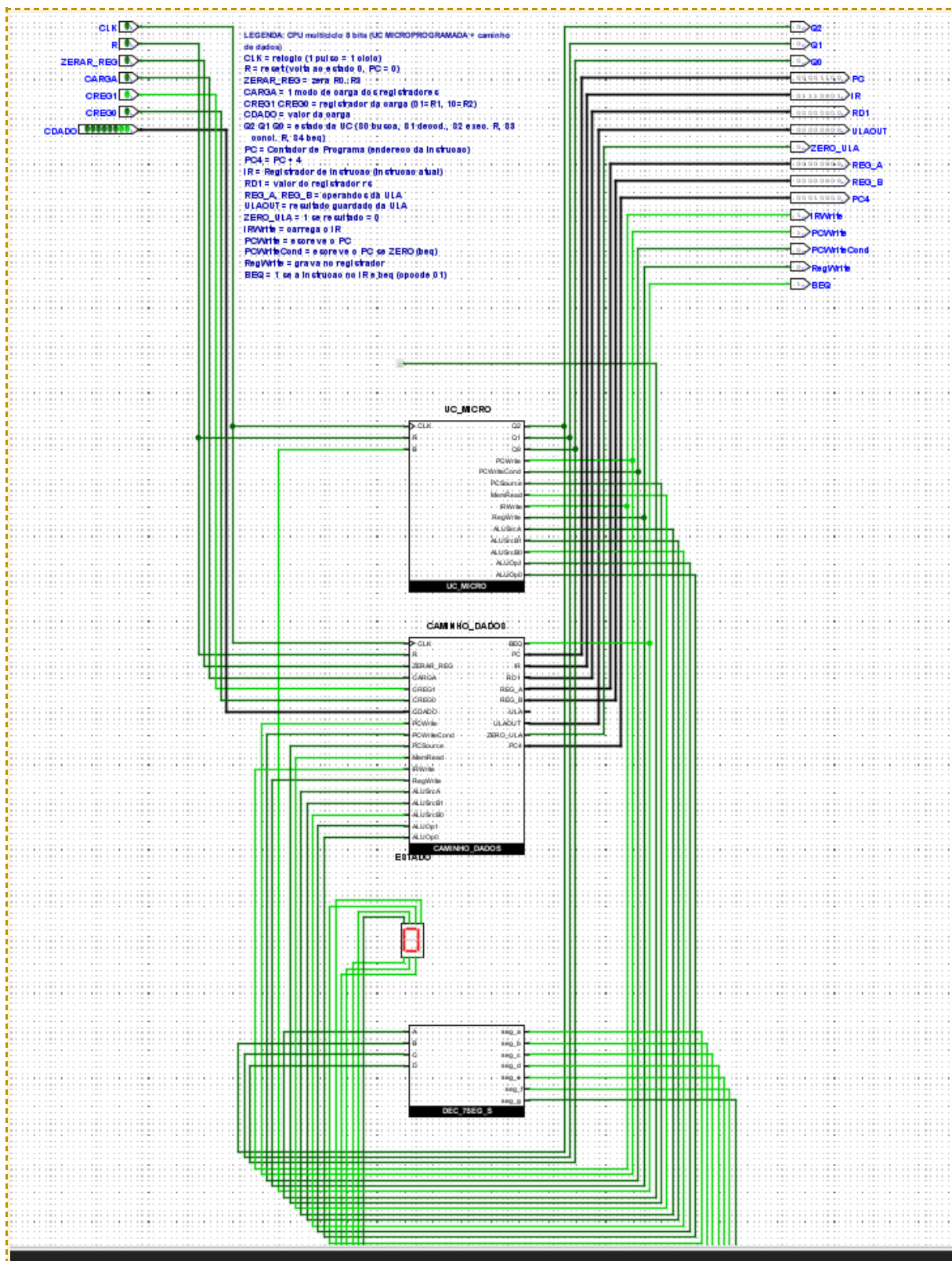
4.7 Microprograma

A versão microprogramada guarda os sinais numa ROM de 8 posições por 8 bits. Cada microinstrução tem quatro campos de 2 bits: ULAOp (bits 7–6), Fonte (5–4), Ação (3–2) e Seq (1–0). O μ PC é o CONTADOR_UPC, um contador síncrono de 3 bits com carga (Componente 13), feito com três FF_D e portas. O próximo endereço depende de Seq: 00 $\rightarrow$ μ PC+1; 01 $\rightarrow$ 000 (volta à busca); 10 $\rightarrow$ despacho pelo opcode (B=1 $\rightarrow$ 100, B=0 $\rightarrow$ 010); 11 $\rightarrow$ 000.

Tabela 39 — Microprograma

Rótulo	End.	ULAOp	Fonte	Ação	Seq	Hex
Busca	0	00	00	01	00	0x04
Decod	1	00	10	00	10	0x22
ExecR	2	10	01	00	00	0x90
ConclR	3	10	01	10	01	0x99
ConclBEQ	4	01	01	11	01	0x5D

Em ConclR foi usada Fonte=01, e não 00, porque pela convenção do enunciado Fonte=00 escreve o PC, o que quebraria o programa; repetir ULAOp=10 e Fonte=01 mantém o mesmo valor em ULAOut. A decodificação dos campos em sinais é: ALUOp = ULAOp; ALUSrcA = F0; ALUSrcB1 = F1; ALUSrcB0 = PCWrite = F1'·F0'; IRWrite = MemRead = A1'·A0; RegWrite = A1·A0'; PCWriteCond = PCSource = A1·A0. O arquivo microcodigo.txt guarda essas cinco palavras no formato de imagem de ROM do Logisim-Evolution (cabeçalho "v3.0 hex words addressed" seguido de "00: 04 22 90 99 5d 00 00 00"); é esse arquivo que é carregado na ROM da UC_MICRO. As posições 5 a 7 ficam livres.



separados para dois registradores de leitura, um registrador de destino, o funct e um imediato, foram feitas duas simplificações. Nas instruções tipo-R, o registrador de destino é o próprio rs ($rd = rs$): add R1, R2 faz $R1 = R1 + R2$. No beq, o imediato de 4 bits ocupa os bits 3–0 e se sobrepõe aos campos rt e funct, de modo que os bits 3–2 servem ao mesmo tempo como rt e como parte alta do deslocamento.

Tabela 40 — Formato da instrução de 8 bits

Bits	Campo	Função	Valores
7–6	opcode	tipo da instrução	00 = tipo-R; 01 = beq
5–4	rs (= rd)	1º operando; destino no tipo-R	00 a 11 = R0 a R3
3–2	rt	2º operando	00 a 11 = R0 a R3
1–0	funct	operação do tipo-R	00 add, 01 sub, 10 and, 11 or
3–0	imm	deslocamento do beq	4 bits com sinal (–8 a +7), em instruções

A entrada B da unidade de controle vem do opcode: $B = IR7' \cdot IR6$, que vale 1 somente no beq.

O caminho de dados (subcircuito CAMINHO_DADOS) é o mesmo nas duas CPUs; muda apenas a unidade de controle ligada a ele: a CPU_CABEADA usa a UC_CABEADA e a CPU_MICRO usa a UC_MICRO. Ele contém o PC de 8 bits, que conta em bytes; a ROM de programa 16×8 (Componente 05), lida com os bits $PC[5:2]$, já que cada instrução avança 4 no PC; o registrador de instrução IR; o banco R0–R3 (BANCO_REG, Componente 07), com leitura de rs e rt e escrita em rs; os registradores A, B e ULAOut; a ULA8 (Componente 10), comandada pelo CONTROLE_ULA a partir de ALUOp e funct, com a saída ZERO; e o extensor de sinal EXT_SINAL_4_8 (Componente 11), cuja saída é deslocada duas posições à esquerda (multiplicada por 4) para formar o deslocamento do beq em bytes.

Três MUX4_8 (Componente 02) fazem as escolhas: PCSrc (0 = saída da ULA, 1 = ULAOut), ALUSrcA (0 = PC, 1 = A) e ALUSrcB (00 = B, 01 = constante 4, 10 = imediato estendido $\times 4$). O PC é escrito quando $PCWrite = 1$, ou quando $PCWriteCond = 1$ e $ZERO = 1$. O somador com constante SOMA MAIS_4 (Componente 04) calcula $PC4 = PC + 4$, mostrado no pino PC4 para conferência.

Para dar valores iniciais aos registradores sem precisar de instruções de carga, foi incluído um modo de carga. Com $R = 1$ e $CARGA = 1$, os pinos CREG1 e CREG0 escolhem o registrador (01 = R1, 10 = R2), CDADO dá o valor e cada pulso de CLK grava esse valor. Enquanto $CARGA = 1$, a saída RD1 mostra o registrador escolhido, o que permite conferir a carga. O pino ZERAR_REG zera R0 a R3.

O programa de teste gravado na ROM está na Tabela 41. Com $R1 = 5$, $R2 = 3$ e $R0 = R3 = 0$, ele exercita uma instrução tipo-R e os dois casos do beq: um desvio tomado e um não tomado.

Tabela 41 — Programa de teste gravado na ROM de programa

Endereço	Hex	Instrução	Efeito
0x00	18	add R1, R2	$R1 = 5 + 3 = 8$
0x04	71	beq R3, R0, +1	$R3 = R0 = 0$: desvio tomado, $PC = 0x0C$
0x08	14	add R1, R1	não executada (pulada pelo desvio)
0x0C	51	beq R1, R0, +1	$R1 = 8 \neq R0$: desvio não tomado, $PC = 0x10$

Endereço	Hex	Instrução	Efeito
0x10	19	sub R1, R2	R1 = R1 - R2
0x14	1A	and R1, R2	R1 = R1 AND R2
0x18	1B	or R1, R2	R1 = R1 OR R2
0x1C	4F	beq R0, R3, -1	R0 = R3: desvia para si mesma (fim do programa)

O detector da sequência 101 (DETECTOR_101, Componente 09) está em parte2_cabeada.circ, junto com o circuito de teste TESTE_DETECTOR_101. É uma máquina de Moore com dois flip-flops D que reconhece 101 com sobreposição. Os estados são S0 = 00 (início), S1 = 01 (viu 1), S2 = 10 (viu 10) e S3 = 11 (viu 101, Z = 1), e as equações obtidas são:

$$D1 = X' \cdot Q0 + X \cdot Q1 \cdot Q0' \quad D0 = X \quad Z = Q1 \cdot Q0$$

4.9 Testes da Parte II

Os testes T-06 a T-09 foram feitos à mão no Logisim-Evolution, executando o programa da Tabela 41 pulso a pulso. Antes de cada execução a simulação foi reiniciada (Ctrl+R) e os registradores foram carregados pelo modo de carga com R1 = 5 e R2 = 3 (Figuras 37 e 38). Depois, com CARGA = 0 e R = 0, cada pulso de CLK avança um ciclo. O estado da unidade de controle aparece nos pinos Q2 Q1 Q0 e no display de sete segmentos. A Tabela 42 compara o esperado com o observado na tela em cada pulso; os valores de 8 bits estão em hexadecimal, e nas figuras aparecem em binário.

Tabela 42 — Testes da Parte II (CPU_CABEADA nos testes T-06 a T-08 e CPU_MICRO no T-09)

Teste	Pulso	Esperado	Obtido	Figura
Carga	R1 ← 5	RD1 = 05	RD1 = 05	Figura 37
Carga	R2 ← 3	RD1 = 03	RD1 = 03	Figura 38
T-06	reset (pulso 0)	Q = 000; PC = 00; IRWrite = PCWrite = 1	Q = 000; PC = 00; IRWrite = PCWrite = 1	Figura 39
T-06	pulso 1	Q = 001; PC = 04; IR = 18	Q = 001; PC = 04; IR = 18	Figura 40
T-07	pulso 2	Q = 010; A = 05; B = 03	Q = 010; A = 05; B = 03; display 2	Figuras 41 e 42
T-07	pulso 3	Q = 011; ULAAOut = 08; RegWrite = 1	Q = 011; ULAAOut = 08; RegWrite = 1; display 3	Figuras 43 e 44
T-07	pulso 4	Q = 000; R1 = 08	Q = 000; RD1 = 08; display 0	Figuras 45 e 46
T-08	pulso 6	Q = 100; IR = 71; ZERO = 1; PCWriteCond = 1; ULAAOut = 0C	Q = 100; IR = 71; ZERO = 1; PCWriteCond = 1; ULAAOut = 0C; display 4	Figuras 47 e 48
T-08	pulso 7	Q = 000; PC = 0C (desvio tomado)	Q = 000; PC = 0C	Figura 49
T-08	pulso 9	Q = 100; IR = 51; ZERO = 0; PC fica em 10 (desvio não tomado)	Q = 100; IR = 51; ZERO = 0; PC = 10	Figura 50
T-09	reset (pulso 0)	Q = 000; PC = 00 (CPU_MICRO)	Q = 000; PC = 00	Figura 51
T-09	pulso 7	Q = 000; PC = 0C; IR = 71 (CPU_MICRO)	Q = 000; PC = 0C; IR = 71	Figura 52

No T-07, a instrução add R1, R2 passa pelos estados S0, S1, S2 e S3. Em S2 os operandos já estão em A e B (5 e 3); em S3 a ULA entrega 8 em ULAOut e RegWrite = 1 grava o resultado em R1, que aparece em RD1 no pulso seguinte. No T-08 aparecem os dois casos do beq. Na primeira beq (endereço 0x04), R3 e R0 valem 0, a subtração A - B dá zero e o PC recebe ULAOut = 0x0C, pulando a instrução do endereço 0x08. Na segunda (endereço 0x0C), R1 = 8 e R0 = 0, então ZERO fica em 0 e, mesmo com PCWriteCond = 1, o PC não é escrito e continua em 0x10.

No T-09 o mesmo programa foi executado na CPU_MICRO, com a mesma carga inicial, e passou pela mesma sequência de estados e valores da Tabela 42. As Figuras 51 e 52 mostram o início (pulso 0) e o desvio tomado (pulso 7, PC = 0x0C). Como conferência complementar, o vetor de teste teste_T09_UC.txt compara os onze sinais de controle das duas unidades em 16 ciclos e foi aprovado nas duas versões.

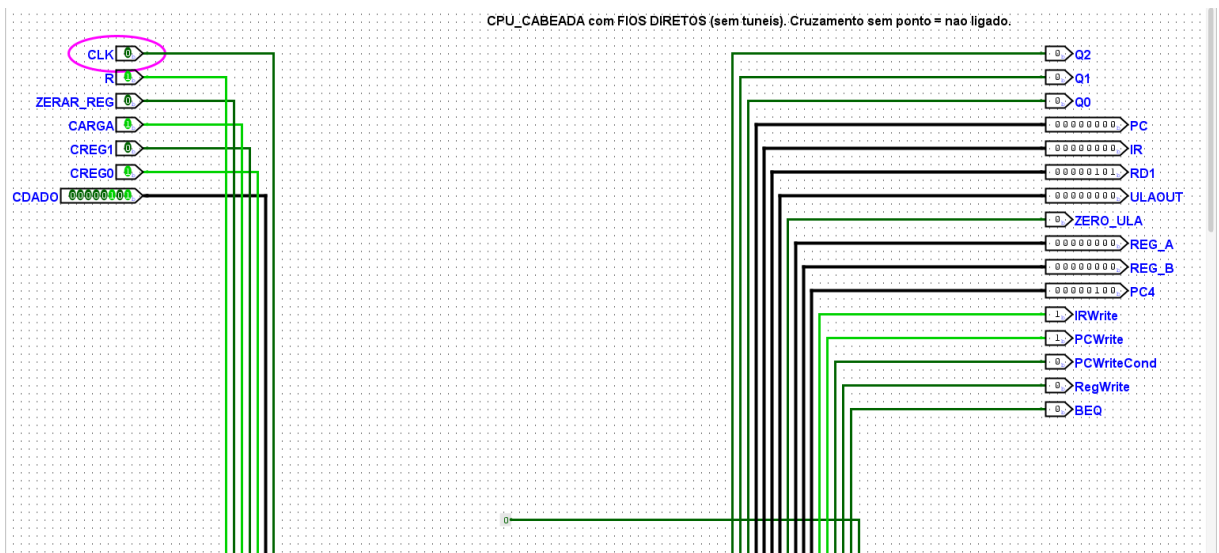


Figura 37 — Preparação: carga de R1 = 5 (R = 1, CARGA = 1, CREG = 01; RD1 = 00000101)

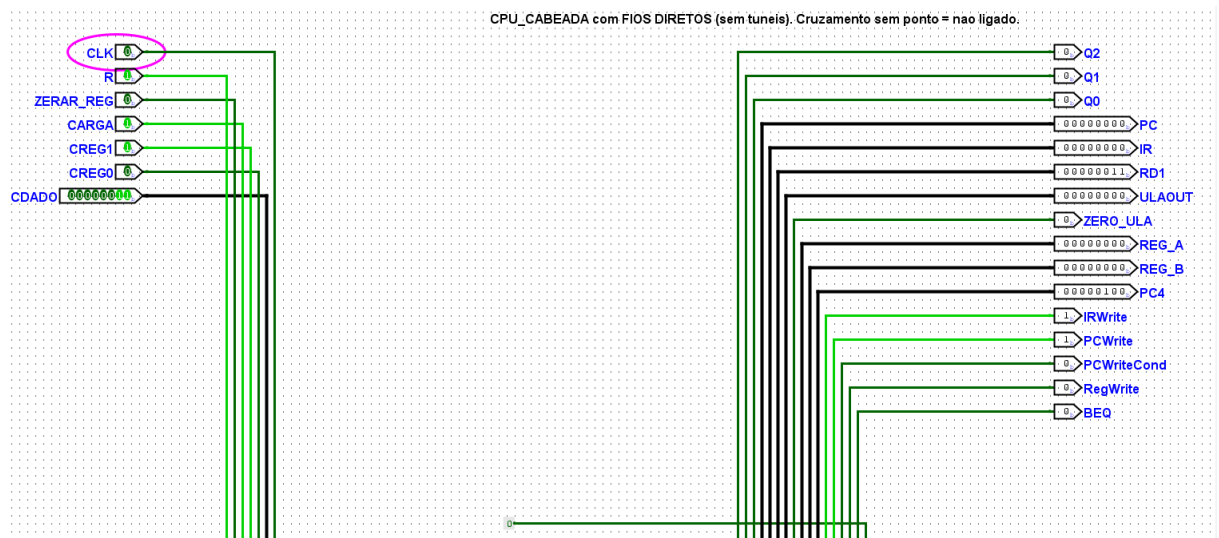


Figura 38 — Preparação: carga de R2 = 3 (CREG = 10; RD1 = 00000011)

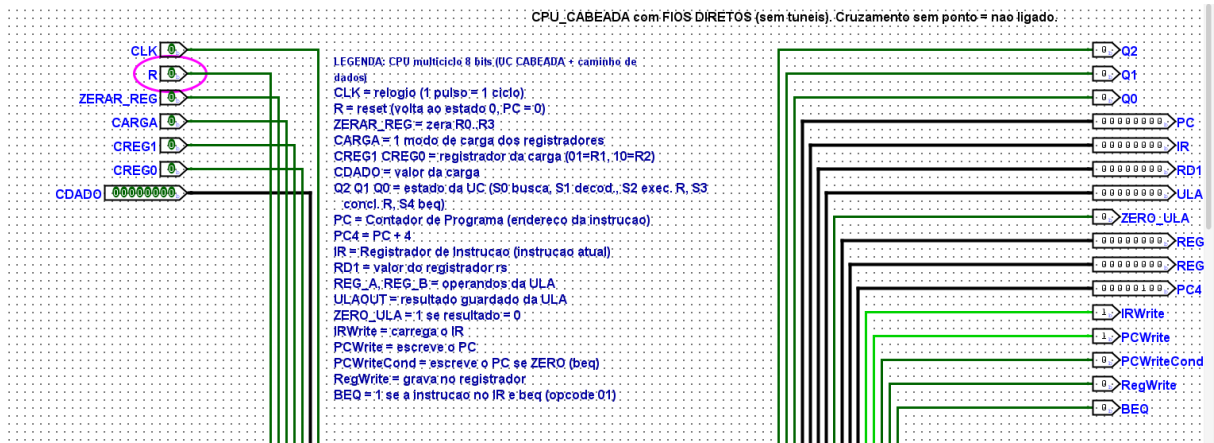


Figura 39 — T-06: estado 0 após o reset (Q = 000, PC = 00, IRWrite = 1)



Figura 40 — T-06: pulso 1, estado 1 com add R1, R2 no IR (IR = 00011000, PC = 00000100)

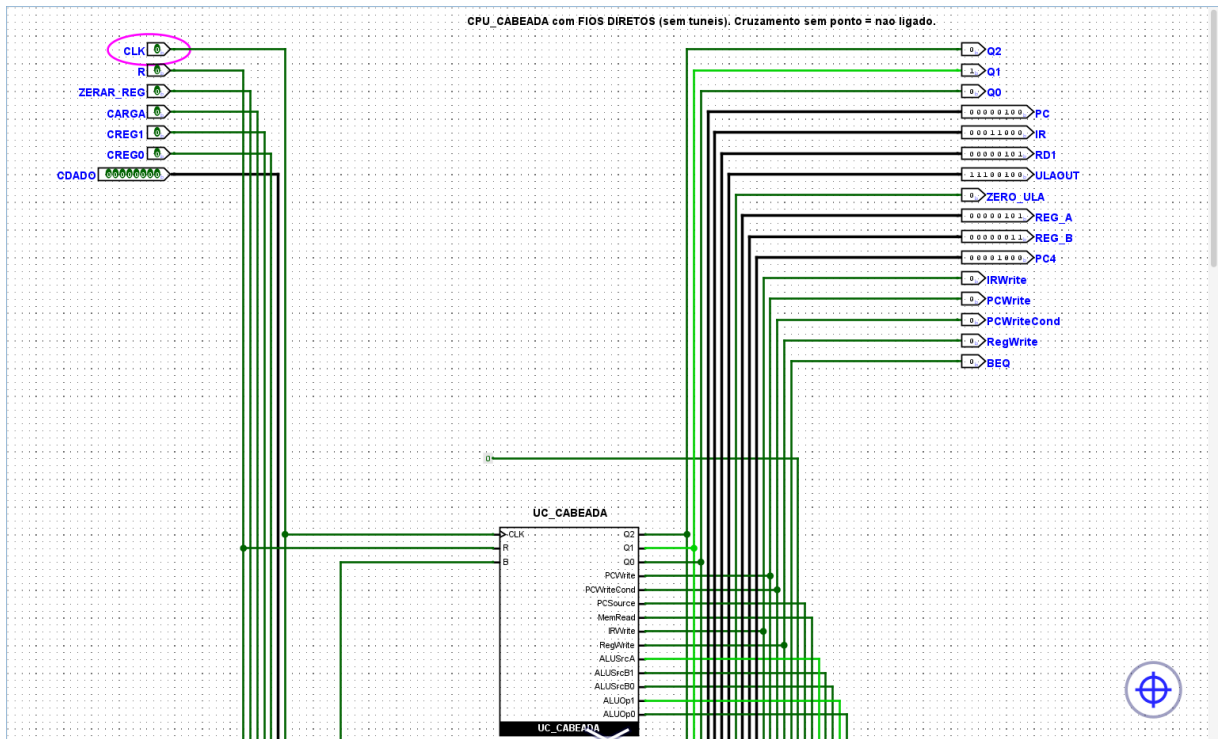


Figura 41 — T-07: pulso 2, estado 2 com os operandos A = 5 e B = 3

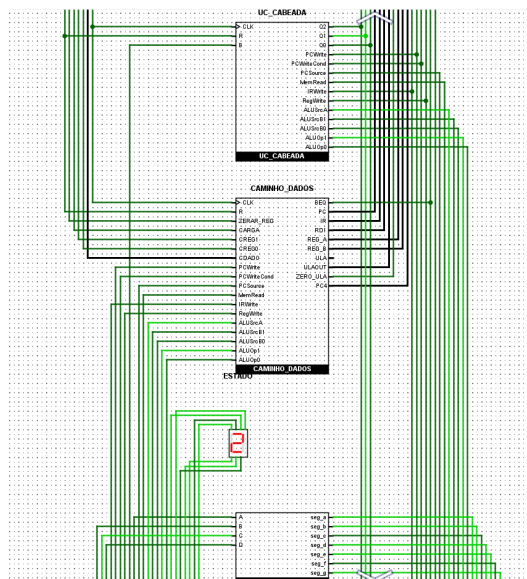


Figura 42 — T-07: pulso 2, UC_CABEADA ligada ao CAMINHO_DADOS e display no estado 2

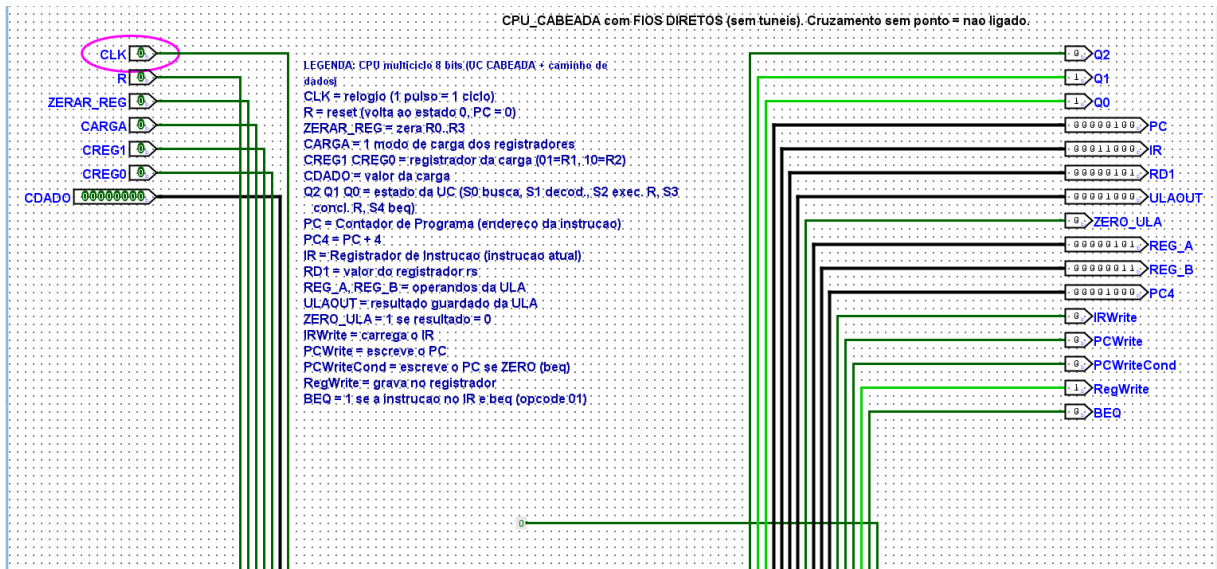


Figura 43 — T-07: pulso 3, estado 3 com ULAOut = 8 e RegWrite = 1

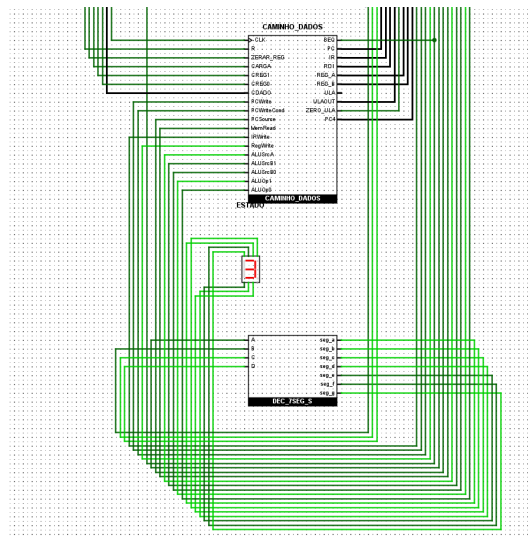


Figura 44 — T-07: pulso 3, display no estado 3

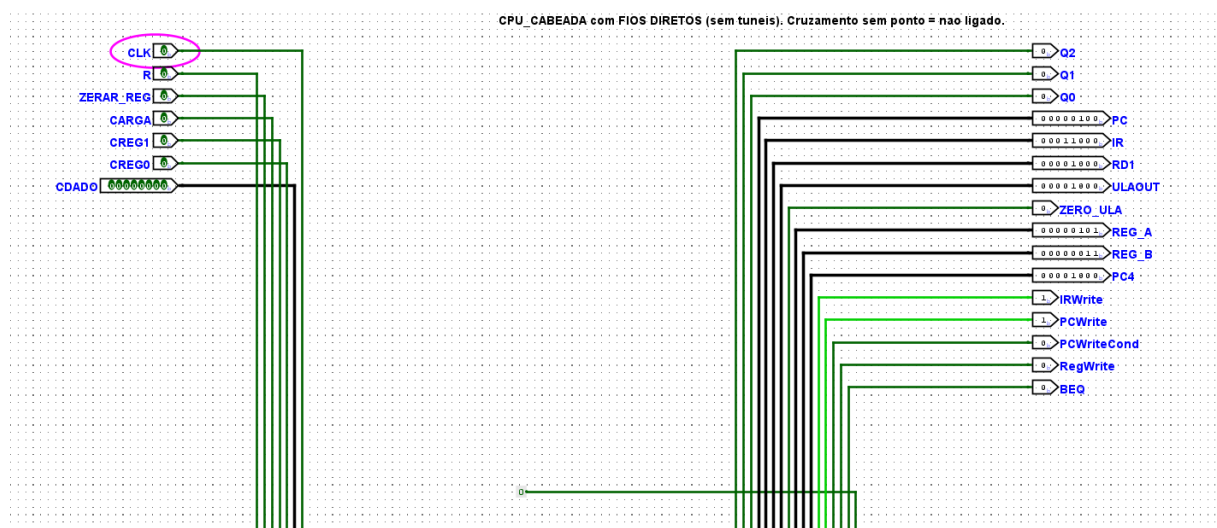


Figura 45 — T-07: pulso 4, de volta ao estado 0 com R1 = 8 (RD1 = 00001000)

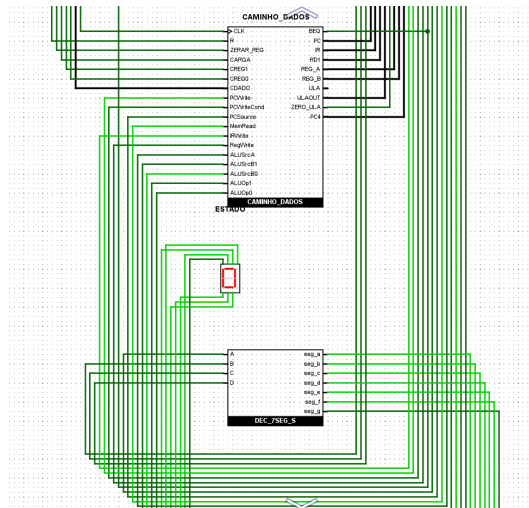


Figura 46 — T-07: pulso 4, display no estado 0



Figura 47 — T-08: pulso 6, estado 4 da primeira beq com ZERO = 1 (desvio tomado)

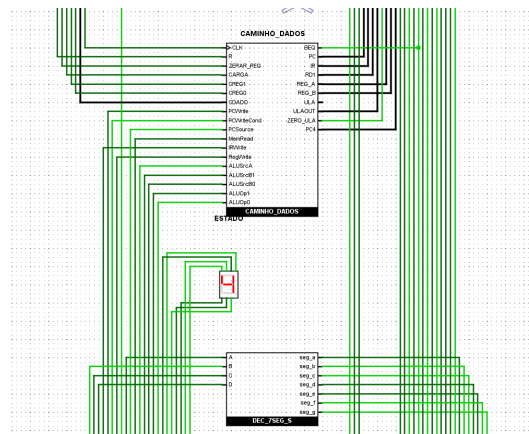


Figura 48 — T-08: pulso 6, display no estado 4

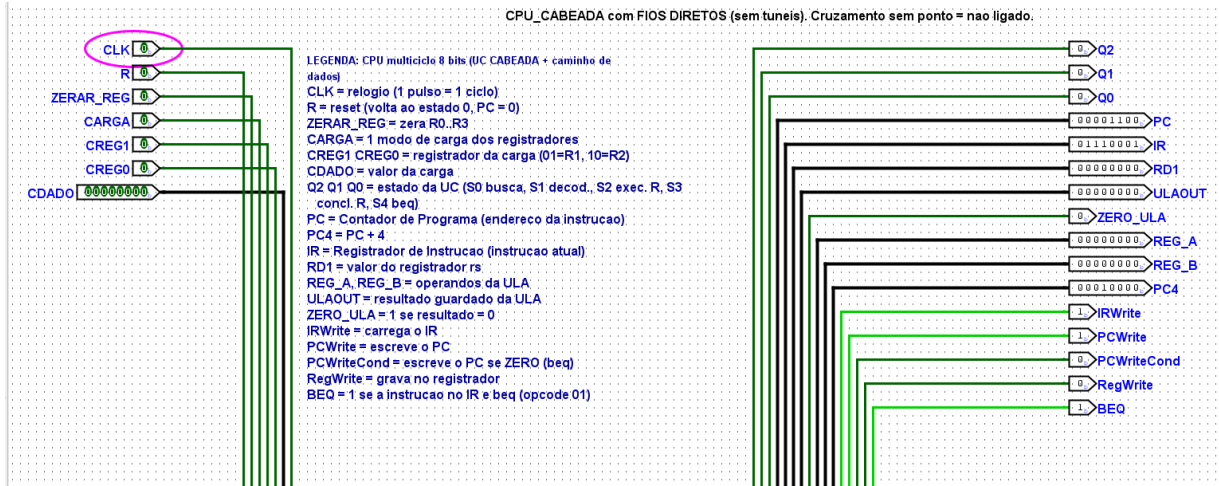


Figura 49 — T-08: pulso 7, PC = 00001100 (0x0C) após o desvio tomado

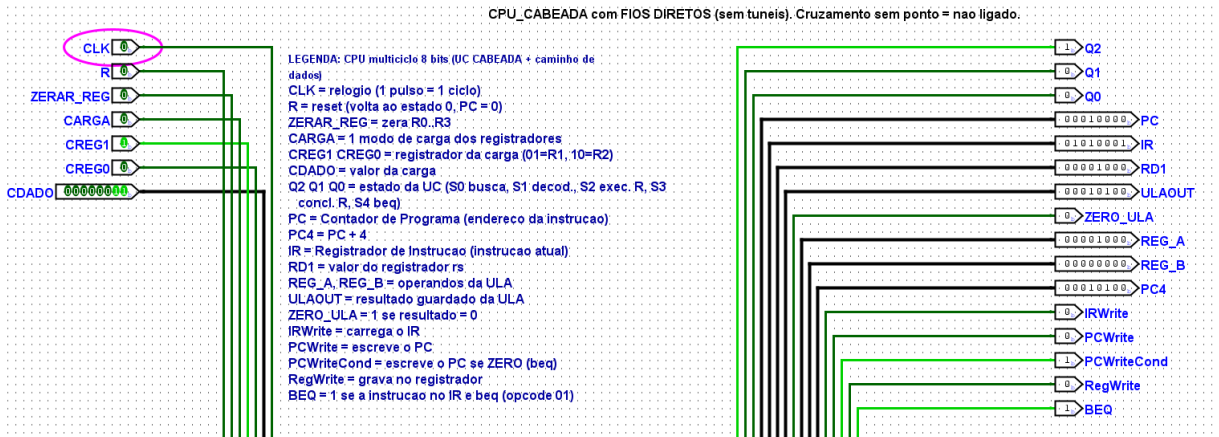


Figura 50 — T-08: pulso 9, segunda beq com ZERO = 0; o PC fica em 00010000 (0x10)

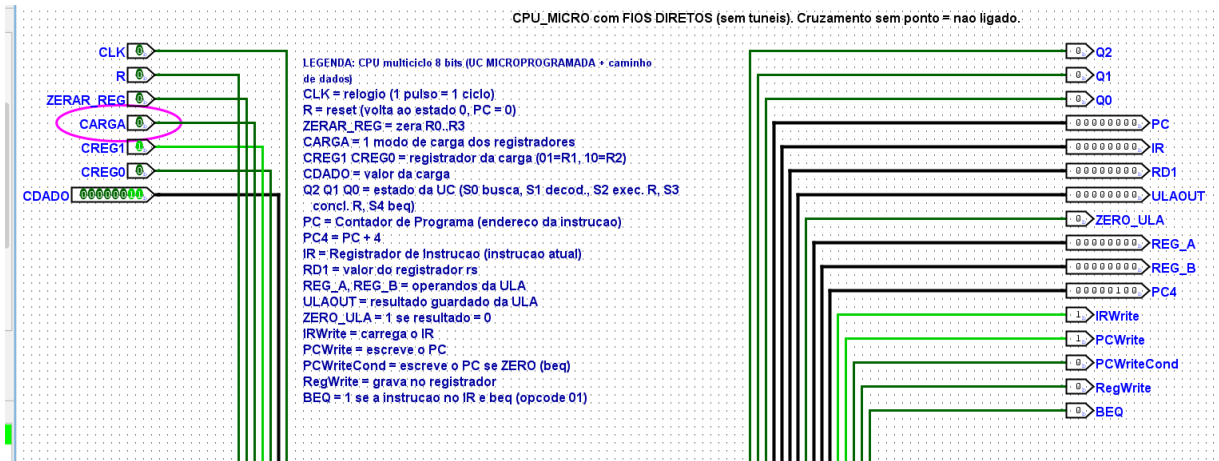


Figura 51 — T-09: CPU_MICRO no estado 0 após a carga (PC = 00)

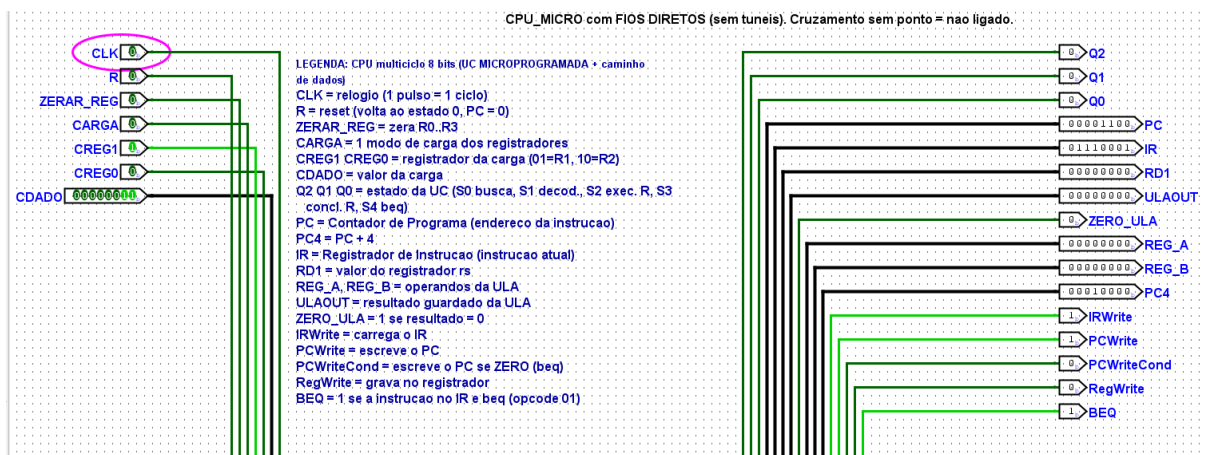


Figura 52 — T-09: CPU_MICRO no pulso 7, PC = 00001100 (0x0C) após o desvio tomado

4.10 Comparação entre as duas versões

A versão cabeada consumiu 13 portas lógicas (4 NOT, 7 AND e 2 OR), mais os três flip-flops D do registrador de estado. A versão microprogramada substituiu a lógica de saída por uma ROM de $8 \times 8 = 64$ bits, dos quais 40 estão ocupados pelas cinco microinstruções, mais a decodificação dos campos e o despacho (7 NOT, 6 AND e 1 OR) e o μ PC CONTADOR_UPC (7 NOT, 9 AND e 3 OR, além de três FF_D). Em número de portas, a microprogramada ficou maior, porque a máquina tem só cinco estados e nove sinais e o μ PC com carga também é feito com portas. A vantagem dela não está no custo, e sim em onde o comportamento fica guardado.

A versão cabeada exigiu mais tempo de projeto. Foi preciso montar a tabela de transição, desenhar sete mapas de Karnaugh, escolher o tratamento dos estados não usados e conferir as equações por simulação. A microprogramada foi mais rápida: depois de definido o formato da microinstrução, bastou escrever cinco palavras de 8 bits, e a correção de um sinal se resume a trocar um valor hexadecimal na ROM, sem refazer portas.

Para acrescentar uma quinta operação à ULA, como as operações tipo-R são distinguidas pelo campo funct e o controle só envia ALUOp=10, nenhuma das três equações de próximo estado precisaria ser refeita na versão cabeada, e nenhuma microinstrução nova seria necessária na microprogramada; a mudança ficaria no controle da ULA. Já uma instrução com fluxo novo, como um acesso à memória, exigiria refazer N0, N1 e N2 e os sinais afetados na versão cabeada, enquanto na microprogramada bastaria escrever uma ou duas microinstruções nas posições livres 5 a 7 e ajustar o despacho.

Em projetos reais, a abordagem cabeada é preferível quando o conjunto de instruções é pequeno e estável e a velocidade é prioritária, como em processadores RISC e controladores embarcados simples, pois não há o atraso de leitura da ROM. A microprogramada é preferível quando o conjunto de instruções é grande ou complexo, ou quando se quer corrigir e estender o comportamento depois de fabricado, como nos processadores CISC, que usam microcódigo atualizável.

5 Declaração de uso de inteligência artificial

Em conformidade com as regras da atividade, a equipe declara o uso da ferramenta de IA generativa Claude (Anthropic) nas seguintes etapas e finalidades:

- **Planejamento:** organização das etapas do laboratório e conferência dos requisitos do enunciado.
- **Projeto dos circuitos:** sugestão de equações e geração de arquivos .circ e de "kits" de montagem para o Logisim-Evolution, que a equipe completou, montou e testou.
- **Verificação:** criação de vetores de teste, usados como conferência complementar aos testes manuais, e checagem por simulação dos circuitos.
- **Recuperação de arquivos:** reconstrução dos subcircuitos perdidos após o fechamento do Logisim sem salvar.
- **Parte II:** geração do caminho de dados e das CPUs completas a partir do formato de instrução escolhido pela equipe, e cálculo dos valores esperados em cada pulso, conferidos pela equipe no simulador.
- **Organização:** legendas explicativas nos circuitos e organização do repositório e das capturas de tela.
- **Documentação:** planilhas de apoio, diagramas, o rascunho deste relatório e a sua atualização com os resultados da Parte II.

Todo o conteúdo foi revisado pela equipe, os testes foram executados no simulador pela equipe e as capturas de tela são das nossas próprias simulações.

6 Divisão do trabalho

Tabela 43 — Divisão do trabalho entre os integrantes

Integrante	Atividades
Rafael da Silva (2021022827)	Parte II: unidade de controle cabeada (tabelas, mapas de Karnaugh e equações), unidade de controle microprogramada e microcódigo, tabela de tempo e CPI, caminho de dados e formato da instrução, testes T-06 a T-09 e organização do repositório.
Jaqueline Braga Meneses (2023010478)	Parte I: componentes base, módulo de memória de 64 bytes com paridade, memória cache de mapeamento direto, planilha do traço e testes T-01 a T-05.

7 Conclusão

A Parte I resultou num módulo de memória de 64 bytes em que cada região responde na faixa correta, com MUX de saída comandado por A5 e A4 e detecção de erro de paridade demonstrada no teste T-02. A cache de mapeamento direto reproduziu no circuito exatamente o resultado previsto na planilha: 8 acertos e 8 faltas no traço, sendo 4 compulsórias e 4 por conflito, com AMAT de 5 ciclos. O teste com o detector de primos mostrou de forma concreta a principal fraqueza do mapeamento direto, em que endereços com o mesmo índice se expulsam mutuamente mesmo havendo linhas livres.

Na Parte II, as unidades de controle cabeada e microprogramada foram ligadas ao mesmo caminho de dados de 8 bits e executaram o programa de teste: a instrução add gravou R1 = 8, o primeiro beq desviou para 0x0C e o segundo não desviou, como previsto. As duas versões passaram pela mesma sequência de estados e valores, o que confirma a equivalência. A comparação mostrou que, para uma máquina tão pequena, a versão cabeada gasta menos portas e tende a ser mais rápida, enquanto a microprogramada é mais fácil de projetar e de modificar. A principal limitação do projeto está no formato de 8 bits, que obrigou a usar $rd = rs$ e a sobrepor o imediato do beq aos campos rt e $funct$.

Referências

PATTERSON, David A.; HENNESSY, John L. Organização e projeto de computadores: a interface hardware/software. 5. ed. Rio de Janeiro: Elsevier, 2017.

STALLINGS, William. Arquitetura e organização de computadores. 10. ed. São Paulo: Pearson, 2017.

LOGISIM-EVOLUTION. Logisim-evolution: digital logic design tool and simulator. Versão 5.0.0. Disponível em: <https://github.com/logisim-evolution/logisim-evolution>.

ROCHA, Herbert Oliveira. Laboratório de Circuitos — Projeto Integrador: subsistema de memória e unidade de controle. Boa Vista: UFRR, 2026. Enunciado da atividade.